Active Learning on Distributions

Complete English Chapters 3–4 with Figures

Based on the English chapter file prepared externally

Contents

Source Figure Note

This wrapper compiles the English chapter body while using figures extracted from the original thesis PDF. The expected figure files are listed in table 1; all files are stored in `figures/` and are referenced by the chapter body with the same filenames.

Table 1: Expected figure manifest for Chapters 3 and 4

File	Source figure	Intended content
fig3_1a.png	Fig. 3.1(a)	Synthetic data/outlier panel
fig3_1b.png	Fig. 3.1(b)	Synthetic data/outlier panel
fig3_1c.png	Fig. 3.1(c)	Synthetic data/outlier panel
fig3_1d.png	Fig. 3.1(d)	Synthetic data/outlier panel
fig3_5.png	Fig. 3.5	Functional gradient approach to PMAL
fig4_1.png	Fig. 4.1	Dataset performance comparison, part 1
fig4_2.png	Fig. 4.2	Dataset performance comparison, part 2
fig4_3.png	Fig. 4.3	Dataset performance comparison, part 3
fig4_4.png	Fig. 4.4	Comparison with boosting methods
fig4_5.png	Fig. 4.5	Comparison with distribution discrepancy methods
fig4_6.png	Fig. 4.6	GAUSSDIST experiment
fig4_7.png	Fig. 4.7	20 Newsgroups experiment, part 1
fig4_8.png	Fig. 4.8	20 Newsgroups experiment, part 2
fig4_9.png	Fig. 4.9	USPSDIST experiment
fig4_10.png	Fig. 4.10	PMARLDS vs PMARLDB experiment

Chapter 1

Active Learning on Distributions

The goal of this thesis is to present active learning on distributions. The lack of direct access to the distribution is a fundamental challenge in the active learning problem. As we mentioned, access to distributions is indirect, through samples drawn from them. Consequently, the accuracy of estimating distributions will be effective in active learning.

In the first part of this chapter, Section 1.0.1, we first examine theoretical approaches for the active learning problem in general. In this section, in 1.0.1.4, we present a method that plays an important role in the continuation and subsequent sections and prove that it is optimal. Then, in the second part, we solve the mentioned method for the active learning problem in the input space using a functional approach. This is a general algorithm that can be used for different sets of functions for active learning. We present a special case of this algorithm based on a linear form for the set of functions and examine its theoretical properties. Now, after preparing the theoretical tools, in the third part, we address the main goal of the thesis, which is the development of an active learning algorithm for distributional data.

1.0.1 Analysis of Theoretical Approaches for the Active Learning Problem

The goal of this section is to present a principled approach for active learning. In learning theory, there are two approaches for classification: Empirical Risk Minimization (ERM) and the Plug-in Classifier. In the following, we first address the development of a method for active learning using ERM in 1.0.1.1, and then in 1.0.1.4, we focus on the plug-in classifier for active learning. In 1.0.1.4, we introduce the Probabilistic Minimax Active Learning (PMAL) method.

1.0.1.1 Towards Active Learning for Empirical Risk Minimization

Assume $L = \{(x_i, y_i) | x_i \in \mathbb{R}^d, y_i \in \{-1, 1\}\}$ is the set of labeled data and $U = \{x_i | x_i \in \mathbb{R}^d\}$ is the set of unlabeled data. If the cost function on this data using a loss function l is defined as

$$J(f, \lambda) = \sum_{i=1}^{n_l} l(y_i, f(x_i)) + \frac{\lambda}{2} \|f\|^2, \tag{1.1}$$

then, based on empirical risk minimization, the optimal classifier in the function set $\mathcal{F}$ will be:

$$f^* = \arg\min_{f \in \mathcal{F}} J(f, \lambda). \tag{1.2}$$

To select a query point based on the version space reduction approach, we should query data that, regardless of its label, has the minimum loss. In other words,

$$x_q^* = \arg\min_{x_q \in U} \max\{l(1, f^*(x_q)), l(-1, f^*(x_q))\} = \arg\min_{x_q \in U} \max_{y_q} l(y_q, f(x_q)). \tag{1.3}$$

Unfortunately, in the early stages of active learning, the current classifier boundary differs significantly from the optimal classifier boundary. Gradually, as the number of labeled samples increases, the difference between the current classifier and the optimal classifier is expected to decrease. The objective function (1.3) is equivalent to the following objective function:

$$x_q^* = \arg\min_{x_q \in U} \min_{\substack{f \in \mathcal{F}, \\ J(f,\lambda) \leq J(f^*,\lambda)}} \max_{y_q} l(y_q, f(x_q)). \tag{1.4}$$

We know that for this constrained optimization problem, $\exists c_q \geq 0$ such that the above problem is equivalent to

$$x_q^* = \arg\min_{x_q \in U} \min_{f \in \mathcal{F}} \max_{y_q} c_q l(y_q, f(x_q)) + J(f, \lambda). \tag{1.5}$$

By swapping the minimization over f and maximization over y_q, we have:

$$x_q^* = \arg\min_{x_q \in U} \max_{y_q} \min_{f \in \mathcal{F}} c_q l(y_q, f(x_q)) + J(f, \lambda). \tag{1.6}$$

By repeating the point selection in active learning using the above objective function, the empirical risk for this problem transforms as follows:

$$\hat{J}(f, \lambda) = \sum_{i=1}^{n} c_i l(y_i, f(x_i)) + \frac{\lambda}{2} \|f\|^2, \tag{1.7}$$

where the coefficient c_i for the initial labeled data is equal to one. Given that the labels of unlabeled data are unknown, the objective function of the active learning problem will be:

$$x_q^* = \arg\min_{x_q \in U} \max_{y_q} \min_{f \in \mathcal{F}} \min_{\substack{y_i, \forall x_i \in U, \\ i \neq q}} \hat{J}(f, \lambda). \tag{1.8}$$

The coefficient c_s, which ensures the equivalence of the two optimization problems (1.4) and (1.5), depends on the value of $f(x_q)$ and the loss of other points. Due to the non-independent and non-identically distributed nature of points selected in active learning, this coefficient cannot be assumed to be the same for all points. Using this coefficient, we can adjust the importance of the empirical risk of other points against the loss value at the point in question. If this coefficient is small, it means the loss at the point in question is not very important. If this coefficient is large, it indicates the importance of the loss at this point and consequently its importance for learning in the problem at hand.

As we know, unlabeled data can harm learning [?]. Furthermore, as discussed in the previous chapter, [?] showed that adding some data to labeled data can increase the classifier error. Therefore, the goal is to nullify the effect of undesirable data in the learning process. Let's call the set of points to be removed D_o. We need a method to estimate it and remove or reduce its influence in the active learning process. Suppose we have a function f_o that returns zero if the data point $x_i \in D_o$ and returns y_i if the data point is noisy:

$$f_o(x_i) = \begin{cases} 0 & i \in D - D_o \\ y_i & i \in D_o \end{cases} \tag{1.9}$$

It is clear that if the classifier f_o evaluates a point as part of the removal data, the loss of that point should not be included in the objective function. Therefore, for the loss of a point to be calculated in the above optimization problem, the function f_o must be equal to zero. Consequently, we can set the coefficient c_i equal to the following loss for the function. That is,

$$c_i = \begin{cases} 1 & y_i f_o(x_i) = 0 \\ 0 & y_i f_o(x_i) = 1 \end{cases} \tag{1.10}$$

Unfortunately, the removal data, i.e., the data that distorts the learning process, is unknown. Despite the unknown nature of this data, how can we remove them from the learning and active learning process?

Our main assumption is that for the function f_o to correctly identify this data, it must be much more complex than the function f. To clarify this, we need to refer to the equivalence between regularization and distortion in data points [?]. We know that tolerating more distortion in data points by the function is equivalent to it being more regularized [?]. If we create a large distortion in the independent variable at data points and the function remains unchanged, we are dealing with a simple function. Conversely, removal data are data points where small changes in them cause large changes in the classifier. These data do not follow the geometry of the distribution, and consequently, based on the Manifold Assumption, removing them will improve learning [?]. Therefore, by determining appropriate properties for the function f_o, we can make it identify the data points harmful to learning. First, let's examine this classifier, which we call the Simple-Complex Classifier.

Simple-Complex Classifier The first necessary property for f_o is that it be much more complex than the function f. The second property is the boundedness of the sum of its absolute values over all data points, or in other words, its ℓ_1-norm. In fact, we have assumed that the data unsuitable for learning form a small subset of size n_o of the data. Let $f(x) = w^T\phi(x), w \in \mathcal{H}$, $f_o(x) = w_o^T\psi(x), w_o \in \mathcal{H}_o$. Then, assuming the Hinge loss function $l(y_i, w^T\phi(x_i)) = (1 - y_i w^T\phi(x_i))_+$, and $\Psi(X) = [\psi(x_1), \ldots, \psi(x_n)]$ and $l_{io} = (1 - y_i w_o^T\psi(x_i))_+$, the target classification problem transforms as follows:

$$\bar{J}(w, w_o) = \sum_{i\in D}(1 - y_i w_o^T\psi(x_i))_+(1 - y_i w^T\phi(x_i))_+ + \frac{\lambda}{2}\|w\|^2_{\mathcal{H}} + \frac{\lambda_o}{2}\|w_o\|^2_{\mathcal{H}_o}, \tag{1.11}$$

where the function w_o is much more complex than the function w, using the properties of the Hilbert space $\mathcal{H}_o$ and the parameter λ_o. Therefore, the resulting optimization problem for learning the simple-complex classifier is:

$$\min_{w,w_o} \bar{J}(w, w_o) \tag{1.12a}$$

$$\text{s.t. } \|\Psi(X)^T w_o\|_1 \leq n_o. \tag{1.12b}$$

This problem can also be written as:

$$\min_{w,w_o} \bar{J}(w, w_o) + c_p\|\Psi(X)^T w_o\|_1. \tag{1.13}$$

Problem (1.12) can be equivalently written as:

$$\min_{w,w_o,p} \bar{J}(w,w_o) \tag{1.14a}$$
$$\text{s.t. } \forall i, \quad |w_o^T\phi(x_i)| \le p_i \tag{1.14b}$$
$$p^T\mathbf{1} \le n_o, \quad p_i \in [0,1] \tag{1.14c}$$
$$\lambda \gg \lambda_o. \tag{1.14d}$$

For simplicity, we can assume that the two spaces $\mathcal{H}$ and $\mathcal{H}_o$ are identical.

Theorem 1.1. *Problem* (1.12) *is equivalent to the following problem:*

$$\min_{w_o,p}\max_{\alpha}\sum_{i\in D}\alpha_i(1-y_iw_o^T\phi(x_i))_+ - \frac{1}{2\lambda}\alpha^TYK\odot GY\alpha + \frac{\lambda^o}{2}\|w_o\|^2 \tag{1.15a}$$
$$s.t.\ 0 \le \alpha \le 1 \tag{1.15b}$$
$$|\Phi(X)^Tw_o| \le p, \quad p \le 1, \quad p^T\mathbf{1} \le n_o \tag{1.15c}$$
$$G \succeq (1-Y\Phi(X)^Tw_o)(1-Y\Phi(X)^Tw_o)^T \tag{1.15d}$$
$$\operatorname{diag}(G) = 1 - Y\Phi(X)^Tw_o \tag{1.15e}$$
$$\operatorname{rank}(G) = 1, \tag{1.15f}$$

where $Y = \operatorname{diag}(y)$.

In the case of unlabeled data, due to their unknown labels, the rank constraint on matrix G makes the above problem non-convex. The following theorem presents a convex approximation of this problem.

Theorem 1.2. *Let* $g' = 1 - Y\Phi(X)^Tw_o$. *The convex approximation of the above problem is:*

$$\min_{y_u}\min_{w_o,p,G}\min_{\beta',\eta'} t + \frac{\lambda_o}{2}\|w_o\|^2 + \beta'^T\mathbf{1} \tag{1.16a}$$
$$s.t. \begin{bmatrix} K\odot H & g'+\eta'-\beta' \\ (g'+\eta'-\beta')^T & \frac{2}{\lambda}t \end{bmatrix} \succeq 0 \tag{1.16b}$$
$$\beta',\eta' \ge 0 \tag{1.16c}$$
$$|\Phi(X)^Tw_o| \le p, \quad p \le 1, \quad p^T\mathbf{1} \le n_o \tag{1.16d}$$
$$\begin{bmatrix} H & Y\mathbf{1}-\Phi(X)^Tw_o \\ (Y\mathbf{1}-\Phi(X)^Tw_o)^T & 1 \end{bmatrix} \succeq 0 \tag{1.16e}$$
$$\operatorname{diag}(H) = 1 - \Phi(X)^Tw_o. \tag{1.16f}$$

The proof of these theorems is provided in the appendix.

A point that might seem concerning is that multiplying the loss by a coefficient might exacerbate sample bias in active learning. According to Theorem 3.2.2, because the coefficient of the loss function is bounded, the simple-complex method is consistent; meaning that as the number of labeled examples increases, it converges to the optimal classifier.

In the appendix, we have proven important properties for the simple-complex classifier. Examining the details of these results is beyond the scope of this thesis, and therefore we will not address them. However, as an example, we have proven that if there is outlier data and the model parameters satisfy simple conditions, the simple-complex classifier will eliminate its influence on classification. Furthermore, we have proven that under certain conditions, the simple-complex classifier does not exacerbate bias.

The simple-complex classifier is related in some aspects to learning with a reject option [?]. In this type of learning, points on both sides of the boundary that increase the empirical risk are rejected (removed), and the classification algorithm reduces or eliminates their influence on learning. As we have seen, the development of the above learning algorithm was, of course, independent of its connection to this type of learning algorithm.

During the final stages of this work, a paper by Corrina Cortes and Mehryar Mohri on the learning with a reject option problem was first uploaded to their personal website and then a few months later presented in [?]. Although this paper solely presents a classifier for this purpose, its approach is similar to the approach presented in this section of this thesis, in that it uses an auxiliary function to find rejected data. However, none of the approaches proposed in this thesis for convexifying the problem were presented in that paper. The research presented in this part of the thesis was submitted to arXiv around the same time [?].

Interestingly, the authors in [?] stated that, based on the results of [?], they expected rejected points to be near the classifier boundary, but they expressed surprise that their method also rejected points in regions far from the classifier boundary. As shown in the proofs in the appendix, the proposed method here removes both types of noisy data, i.e., outliers and label noise, from the learning process. Interestingly, that paper stated that algorithms based on learning with a reject option could be used to develop solutions for other learning problems, such as active learning. However, nearly four years after that date and the authors' call for development of such algorithms at an ICML workshop, this has not yet been done for active learning.

Before continuing, it is interesting to observe the results of the simple-complex classifier on synthetic data.

Examining Results on Synthetic Data The simple-complex classifier was introduced to reduce the influence of noisy data, i.e., data that has a negative impact on learning. To study the behavior of the algorithm, we created several synthetic datasets. We expect the simple classifier to be minimally affected by this data, while the complex classifier should identify these examples. In the first dataset, we added outlier data. For clarity, we have added very obvious outliers to the data, the results of which are shown in Figure 3.1. As can be seen, the complex classifier has identified the outliers, and the contour of the simple classifier has received little influence from the outliers. In contrast, the SVM contour has changed relatively. It might seem that because the SVM boundary hasn't changed, this is not very important. To clarify, in Figure 3.1, label noise has been added on both sides of the boundary. As observed, in this case, the SVM boundary has become severely distorted, while the simple classifier boundary has hardly changed. Furthermore, the complex classifier has correctly identified the label noise. When label noise data is far from the classifier boundary, it is natural that its impact on the SVM classifier should be much greater. You can see this case in Figure 3.1. However, the impact of this data on the simple classifier is almost zero. This is thanks to adding a function, the complex classifier, to identify data harmful to learning. Figure 3.1 shows the behavior of the algorithm in a case where outliers are near the boundary.

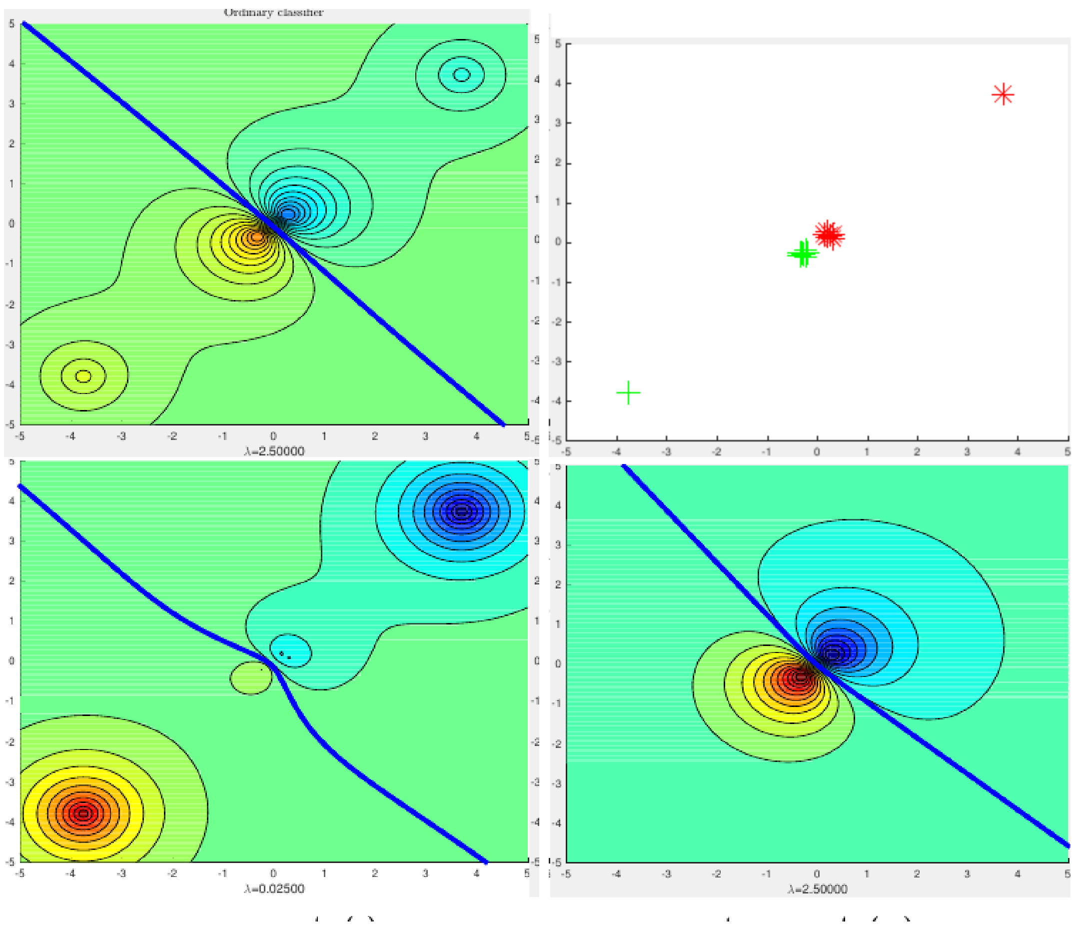


Figure 1.1: Synthetic dataset with clear outliers added to show the contours of the functions. As seen, the simple classifier is relatively unaffected by the outliers, and the complex classifier has absorbed all the outliers. In comparison, the SVM contour is severely affected. This affects the classifier boundary in active learning selection.

Convex Approximation for Robust Active Learning Now we present the convex approximation for active learning. Because the set of query data Q is a subset of the data, we use an integer variable $q_i \in \{0, 1\}$ for this selection, such that if $q_i = 1$, the i-th data point is a member of the query set, and otherwise not. Therefore, the objective function will be:

$$\bar{J}(w, w_o) = \sum_{i \in D}(1-q_i)(1-y_i w_o^T \psi(x_i))_+ (1-y_i w^T \phi(x_i))_+ + \sum_{j \in Q} q_j |w^T \phi(x_j)| + \frac{\lambda}{2}\|w\|_{\mathcal{H}}^2 + \frac{\lambda_o}{2}\|w_o\|_{\mathcal{H}_o}^2. \tag{1.17}$$

Consequently, the objective function of active learning will be:

$$x_s^* = \arg\min_{q} \min_{w, w_o} \min_{\substack{y_i \in \{-1,1\}, \\ \forall x_i \in U}} \bar{J}(w, w_o, q) \tag{1.18a}$$

$$\text{s.t. } \mathbf{1}^T q = b, \quad q_i \in \{0, 1\} \tag{1.18b}$$

$$\|\Psi(X)^T w_o\|_1 \leq n_o. \tag{1.18c}$$

This problem is very difficult. The variables $q_i, y_i, \forall x_i \in U$ are integers. Furthermore, the problem is highly non-convex. We will attempt to approximate this problem in stages with a simpler problem.

Theorem 1.3. *The above problem is equivalent to:*

$$\min_{q,w_o,p}\ \min_{\substack{y_i\in\{-1,1\},\\ \forall x_i\in U}}\ \max_{\alpha} \sum_{i\in D} \alpha_i(1-g_i) + \frac{\lambda_o}{2}\|w_o\|^2 - \frac{1}{2\lambda}\alpha^T K \odot G_b \alpha \tag{1.19a}$$

$$s.t.\ 0 \le \alpha \le 1, \quad -1 \le \tau \le 1, \quad y_i \in \{-1,1\}, \forall x_i \in U \tag{1.19b}$$

$$G_b = \left[\frac{Y(1-g)}{\sqrt{q}}\right]\left[\frac{Y(1-g)}{\sqrt{q}}\right]^T \tag{1.19c}$$

$$g = 1 - (I - \mathrm{diag}(q))(1 - Y\Phi(X)^T w_o) \tag{1.19d}$$

$$|\Phi(X)^T w_o| \le p, \quad p_i \in \{0,1\}, \quad \mathbf{1}^T p \le n_o. \tag{1.19e}$$

The proof of this theorem is provided in the appendix.

This problem is non-convex due to the presence of G_b. Here, we present a convex approximation of the problem. First, consider it in the simple case without w_o and p. In this case, the above problem transforms into the following problem:

$$\min_{q,y_u}\max_{\alpha} \sum_{i\in D} \alpha_i(1-g_i) - \frac{1}{2\lambda}\alpha^T K \odot G_b \alpha \tag{1.20a}$$

$$\text{s.t. } 0 \le \alpha \le 1, \quad -1 \le \tau \le 1, \quad y_i \in \{-1,1\}, \forall x_i \in U \tag{1.20b}$$

$$G_b = \left[\frac{Y(1-q)}{\sqrt{q}}\right]\left[\frac{Y(1-q)}{\sqrt{q}}\right]^T \tag{1.20c}$$

$$g = q \tag{1.20d}$$

$$q_i \in \{0,1\}, \quad \sum_{i\in Q} q_i = b. \tag{1.20e}$$

The constraint related to the matrix G_b is highly non-convex. In [**?**], a convex approximation for the satisfiability problem is proposed, which we use here. For this purpose, we replace the constraint $G_b = \left[\frac{Y(1-q)}{\sqrt{q}}\right]\left[\frac{Y(1-q)}{\sqrt{q}}\right]^T$ with the constraints

$$G_b \succeq \left[\frac{Y(1-q)}{\sqrt{q}}\right]\left[\frac{Y(1-q)}{\sqrt{q}}\right]^T \tag{1.21a}$$

$$\mathrm{diag}(G_b) = \left[\frac{1-\hat{q}}{\hat{q}}\right], \quad \forall i \in D_l, \hat{q}_i = 0, \quad \forall i \in D_u, \hat{q}_i = q_i \tag{1.21b}$$

$$\mathrm{rank}(G_b) = 1. \tag{1.21c}$$

Now, by removing the rank-one constraint on matrix G_b, the convex approximation is obtained. Of course, in this problem, we face another challenge that makes the resulting matrix non-convex. Based on Schur's theorem, we will have:

$$\begin{bmatrix} \hat{G} & y_l & Y_u(1-q) \\ y_l^T & q & 1 \\ (1-q)^T Y_u & 1 & q^T \end{bmatrix} \succeq 0 \tag{1.22}$$

$$\mathrm{diag}(\hat{G}) = 1 - q, \quad \forall i \in D_l, q_i = 0. \tag{1.23}$$

Unfortunately, this constraint is still non-convex due to the term $v_u = Y_u(1-q)$. Fortunately, given that $Y_{u_i} \in \{-1,1\}$, we can represent it as follows:

$$\begin{bmatrix} \hat{G} & y_l & v_u \\ y_l^T & q & 1 \\ v_u^T & 1 & q^T \end{bmatrix} \succeq 0 \tag{1.24}$$

$$|v_u| + q = 1, \quad \mathrm{diag}(\hat{G}) = 1 - q, \quad \forall i \in D_l, = 0. \tag{1.25}$$

By relaxing the discrete values $q_i \in [0,1]$ and $y_i \in [-1,1], \forall x_i \in U$, we have:

$$\min_{q, y_u, G} \max_{\alpha} \sum_{i \in D} \alpha_i (1 - q_i) - \frac{1}{2\lambda} \alpha^T K \odot \hat{G}_b \alpha \tag{1.26a}$$

$$\text{s.t. } 0 \leq \alpha_e \leq 1, \quad -1 \leq \tau \leq 1, \quad y_i \in \{-1, 1\}, \forall x_i \in U \tag{1.26b}$$

$$G = \begin{bmatrix} \hat{G} & y_l & r \\ y_l^T & q & 1 \\ r^T & 1 & q^T \end{bmatrix} \succeq 0 \tag{1.26c}$$

$$|r| + q = 1 \tag{1.26d}$$

$$\mathrm{diag}(\hat{G}_b) = 1 - q, \quad \forall i \in D_l, q_i = 0 \tag{1.26e}$$

$$\mathbf{1}^T q = b, \quad q \in [0,1]. \tag{1.26f}$$

Now we solve the main form of the problem as follows. This problem has greater challenges. In addition to the problems of the previous problem, we have a more complex objective function here. This problem is equivalent to:

$$\min_{q, y_u, w_o, p} \max_{\alpha} \sum_{i \in D} \alpha_i (1 - g_i) + \frac{\lambda_o}{2} \|w_o\|^2 - \frac{1}{2\lambda} \alpha^T K \odot G_b \alpha \tag{1.27a}$$

$$\text{s.t. } 0 \leq \alpha \leq 1, \quad -1 \leq \tau \leq 1, \quad y_i \in \{-1, 1\}, \forall x_i \in U \tag{1.27b}$$

$$G_b = \left[\frac{Y(1-g)}{\sqrt{q}} \right] \left[\frac{Y(1-g)}{\sqrt{q}} \right]^T \tag{1.27c}$$

$$1 - g = (I - \mathrm{diag}(q))(1 - Y \Phi(X)^T w_o) \tag{1.27d}$$

$$|\Phi(X)^T w_o| \leq p, \tag{1.27e}$$

$$\mathbf{1}^T p \leq n_o, \quad p_i \in \{0, 1\} \tag{1.27f}$$

$$\mathbf{1}^T q = b, \quad q_i \in \{0, 1\}. \tag{1.27g}$$

We replace the constraint $G_b = \left[\frac{Y(1-g)}{\sqrt{q}} \right] \left[\frac{Y(1-g)}{\sqrt{q}} \right]^T$ with:

$$G_b \succeq \left[\frac{Y(1-g)}{\sqrt{q}} \right] \left[\frac{Y(1-g)}{\sqrt{q}} \right]^T \tag{1.28a}$$

$$\mathrm{diag}(G_b) = \left[\frac{1-g}{q} \right] \tag{1.28b}$$

$$\mathrm{rank}(G_b) = 1. \tag{1.28c}$$

Based on Schur's theorem, we will have:

$$\begin{bmatrix} \hat{G} & v_l & v_u \\ v_l^T & q & 1 \\ v_u^T & 1 & q^T \end{bmatrix} \succeq 0, \tag{1.29}$$

where

$$v_u = Y_u \, \mathrm{diag}(1 - Y_u \Phi(X_u)^T w_o)(1 - q) \tag{1.30}$$

$$|v_u| = \mathrm{diag}(1 - Y_u \Phi(X_u)^T w_o)(1 - q) \tag{1.31}$$

$$= 1 - Y_u \Phi(X_u)^T w_o - q + \mathrm{diag}(q) Y_u \Phi(X_u)^T w_o \tag{1.32}$$

$$v_l = Y_l \, \mathrm{diag}(1 - Y_l \Phi(X_l)^T w_o). \tag{1.33}$$

It is clear that we do not want to select noisy data for querying. In an ideal state, only one of the two values p_i or q_i will be equal to one, and the other will be zero. Therefore, we impose the constraint $|\Phi(X_u)^T w_o Y_u|$
$le1 - q$. Under these conditions, we can use the following approximation. Given that at least one of the values p_i or q_i is small, the following approximation will be appropriate:

$$|v_u| + q + Y_u \Phi(X_u)^T w_o = 1 \tag{1.34}$$

$$|\Phi(X_u)^T w_o| + qle1. \tag{1.35}$$

Constraint $|v_u| + q + Y_u \Phi(X_u)^T w_o = 1$ is still non-convex. Due to the complexity of the function w_o, we can assume $p_i = Y_u \Phi(X_u)^T w_o$.

The final form for solving this problem is:

$$\min_{G,q,y_u,w_o,p} \max_{\alpha} \sum_{i \in D} \alpha_{e_i}(1 - g_i) + \frac{\lambda_o}{2}\|w_o\|^2 - \frac{1}{2\lambda}\alpha^T(K \odot \hat{G})\alpha + c_a \mathbf{1}^T a \tag{1.36a}$$

$$\text{s.t. } 0 \leq \alpha_e \leq 1, \quad -1 \leq \tau \leq 1, \quad q, p \in [0, 1] \tag{1.36b}$$

$$G = \begin{bmatrix} \hat{G} & v_l & v_u \\ v_l^T & q & 1 \\ v_u^T & 1 & q^T \end{bmatrix} \succeq 0 \tag{1.36c}$$

$$\operatorname{diag}(\hat{G}_b) = \begin{bmatrix} 1 - g \\ \hline q \end{bmatrix}, \quad g = 1 - |v|, \quad |v| \leq a \tag{1.36d}$$

$$a + q + p = 1, \quad \mathbf{1}^T q = b \tag{1.36e}$$

$$|\Phi(X_u)^T w_o| \tag{1.36f}$$

$$le1 - q \tag{1.36g}$$

$$|\Phi(X)^T w_o| \leq p \tag{1.36h}$$

$$\mathbf{1}^T p \leq n_o, \quad \sum_{i, \forall x_i \in L} p_i \tag{1.36i}$$

$$len_l. \tag{1.36j}$$

1.0.1.2 Multi-Task Active Learning

In this section, we propose a method for multi-task active learning based on the empirical risk minimization approach. Assume we have T tasks and the sets

$$L_t = \{(x_{it}, y_{it}) | x_{it} \in \mathbb{R}^d, y_{it} \in \{-1, 1\}\}, \quad \forall t \in [1, \ldots, T] \tag{1.37}$$

are the labeled data sets for the t-th task, and

$$U_t = \{x_{it} | x_{it} \in \mathbb{R}^d\}, \quad \forall t \in [1, \ldots, T] \tag{1.38}$$

are the unlabeled data sets for the same task, and also the labeled and unlabeled data sets

$$D_t = \{(x_{it}, y_{it}) | x_{it} \in \mathbb{R}^d, y_{it} \in \{-1, 1\}\}, \quad \forall t \in [1, \ldots, T] \tag{1.39}$$

where the labels of the unlabeled data are unknown. Also, the set of labeled data for all tasks is $L = \{L_1, \ldots, L_T\}$ and the set of all data is $D = \{D_1, \ldots, D_T\}$. The cost function on the set L, the labeled data for tasks $t \in \{1, \ldots, T\}$, using a loss function l is defined as

$$J(W, L, \lambda) = \sum_{t=1}^{T} \sum_{i=1}^{|L_t|} l(y_{it}, w_t^{*T} x_{it}) + \frac{\lambda}{2}\|W\|_{2,1}, \tag{1.40}$$

where w_t is the t-th column of matrix W, $w_t = W_{*,t}$. Then, based on empirical risk minimization, the optimal tasks will be:

$$W^* = \arg\min_{W} J(W, L, \lambda). \tag{1.41}$$

Similar to before, to select the query point based on the version space reduction approach, we should query a data point that, regardless of its label and task, has the minimum loss. In other words,

$$x_{st}^* = \arg\min_{\substack{x_{st}\in U_t,\\ \forall t,s}} \max_{y_{st}} l(y_{st}, w_t^{*T} x_{st}). \tag{1.42}$$

Similar to the previous section, as the number of labeled samples increases, the difference between the current classifiers of the tasks and the optimal classifier is expected to decrease. Consequently, we will have:

$$x_{st}^* = \arg\min_{\substack{x_{st}\in U_t,\\ \forall t,s}} \min_{\substack{W,\\ J(W,L,\lambda)\leq J(W^*,L,\lambda)}} \max_{y_{st}} l(y_{st}, w_t^{*T} x_{st}). \tag{1.43}$$

We know that for this constrained optimization problem, $\exists c_s \geq 0$ such that the above problem is equivalent to

$$x_s^* = \arg\min_{\substack{x_{st}\in U_t,\\ \forall t,s}} \min_{W} \max_{y_{st}} c_{st} l(y_{st}, w_t^T x_{st}) + J(W, L, \lambda). \tag{1.44}$$

To simplify the problem, we set $c_s = 1$. Consequently, considering the labeled data and the unknown nature of their labels, similar to Section 1.0.1.1, the multi-task active learning problem will be:

$$x_{st}^* = \arg\min_{\substack{x_{st}\in U_t,\\ \forall t,s}} \min_{W} \max_{y_{st}} J(W, D, \lambda). \tag{1.45}$$

In the objective function of this problem, there is a non-differentiable or non-smooth term. To solve this problem, we use the proximal method [**?**]. For simplicity, we use the squared error loss function and set $l(y_{it}, w_t^T \phi(x_{it})) = (y_{it} - w_t^T x_{it})^2$. The objective function (1.45) is maximized when we set the label of the query point to the opposite of the label determined by the classifier; in other words:

$$y_{st} = -\operatorname{sign}(w_t^T x_{st}). \tag{1.46}$$

Consequently, the objective function will be

$$J(W, L, \lambda) = \sum_{t=1}^{T} \sum_{i=1}^{|L_t|} (y_{it} - w_t^T x_{it})^2 + (2 + 2|w_t^T x_{st}|) + \frac{\lambda}{2}\|W\|_{2,1}. \tag{1.47}$$

The term $h(W) = \frac{\lambda}{2}\|W\|_{2,1} + \sum_{t=1}^{T} \sum_{i=1}^{|L_t|} (y_{it} - w_t^T x_{it})^2$ is differentiable, and the term $g(W) = \frac{4}{\lambda} \sum_{t=1}^{T} |w_t^T x_{st}|$ is non-differentiable. Therefore, the solution to the problem is obtained by iterating the following relation with respect to W:

$$W_{k+1} = \operatorname{prox}_{\mu g}\left(W_k - \mu \nabla h(W_k)\right), \tag{1.48}$$

where the proximal operator is defined as

$$\operatorname{prox}_{\mu f}(v) = \arg\min_{x} \left(f(x) + \frac{1}{2\mu}\|x - v\|^2 \right). \tag{1.49}$$

We have:
$$g(W) = \sum_{\substack{u=1 \\ u \neq t}}^{T} \|W_{*,u}\|_2 + \|W_{*,t}\|_2 + \frac{4}{\lambda}|W_{*,t}^T x_{st}|. \tag{1.50}$$
Consequently,
$$\text{prox}_{\mu g}(v) = \arg\min_{W} \left(g(W) + \frac{1}{2\mu}\|W - v\|^2 \right) = \arg\min_{x} \left(\sum_{\substack{u=1 \\ u \neq t}}^{T} \|W_{*,u}\|_2 + \|W_{*,t}\|_2 + \frac{4}{\lambda}|x_{st}^T W_{*,t}| + \frac{1}{2\mu}\|W - \right. \tag{1.51}$$
Because the above problem is separable with respect to the components $W_{*,u}$, we will have the following two problems:
$$\text{prox}_{\mu g}(v)_{*,u,u\neq t} = \arg\min_{W_{u=1,u\neq t}} \left(\sum_{u=1,u\neq t}^{T} \|W_{*,u}\|_2 + \frac{1}{2\mu}\|W_{*,u,u\neq t} - v_{*,u,u\neq t}\|^2 \right), \tag{1.52}$$
$$\text{prox}_{\mu g}(v)_{*,t} = \arg\min_{W_{*,t}} \left(\|W_{*,t}\|_2 + \frac{4}{\lambda}|x_{st}^T W_{*,t}| + \frac{1}{2\mu}\|W_{*,t} - v_{*,t}\|^2 \right). \tag{1.53}$$
For the first problem, we know from [**?**]:
$$\left[\text{prox}_{\mu g}(W)\right]_{*,u,u\neq t} = \left[\left(1 - \frac{\mu}{\|W_{*u}\|_2}\right)_+ W_{*u} \right]_{u=1,u\neq t}^{T}. \tag{1.54}$$
We know that for the absolute value function, the following proximal operator is available:
$$\text{prox}_{\mu|.^T x|}(v) = (v - \mu x)_+ - (-v - \mu x)_+. \tag{1.55}$$
Consequently, for the second problem, we will have:
$$\left[\text{prox}_{\mu g}(W)\right]_{*,t} = \left(1 - \frac{\mu}{\|r\|_2}\right)_+ r, \tag{1.56}$$
where r is equal to:
$$r = \left(W_{*,t} - \mu\frac{4}{\lambda}x_{st}\right)_+ - \left(-W_{*,t} - \mu\frac{4}{\lambda}x_{st}\right)_+. \tag{1.57}$$
Therefore, in the iterative relation (1.48), we have calculated $\text{prox}_{\mu g}(W)$.

The gradient of the function h is:
$$\nabla h(W) = 2X(X^t W - Y). \tag{1.58}$$
Now we have all the components of the iterative relation (1.48). By substituting them into this relation, the solution to the problem is obtained.

1.0.1.3 Learning Using Privileged Information

A student, with the help of a teacher, can learn a concept much faster than a learning machine. In one of his last works, Vapnik posed a question that became the basis for the Learning Using Privileged Information (LUPI) approach: What makes a student (human learner) learn faster than a machine learner? And his answer was: the teacher provides the student with privileged knowledge [?]. The teacher provides the student with a source of knowledge that helps the student learn the concept more accurately and with less effort. Because this knowledge exists only during training and not during testing, Vapnik called it privileged information.

This concept is inherently related to separable/non-separable data in classification. When different classes are separable, the probability of error in predicting the labels of points is of order $O(1/l)$, where l is the number of labeled points. In contrast, in the non-separable case, the error probability is of order $O(1/\sqrt{l})$. This is an enormous difference; for example, for roughly the same accuracy, 320 examples are needed versus 100,000 examples in the non-separable case. The accuracy of machine learning in the non-separable case can be improved using the knowledge provided by the teacher [?].

Assume there is an oracle that can make the data separable again by providing the required slack for each example. The oracle provides the student with a function with infinite VC Dimension that calculates the slack for each example. In this case, the error rate improves to $O(1/l)$. However, such an oracle is not available. Therefore, Vapnik suggested that an Intelligent Teacher provides a triple (x_i, y_i, x_i^*) for each example, where (x_i, y_i) is the example-label pair and x_i^* is the privileged knowledge. The privileged knowledge x_i^* resides in the space X^*, where calculating the slack or predicting the label is much simpler [?]. In other words, the function that calculates the slack is much simpler. This approach is called Learning Using Privileged Information (LUPI).

Almost always, privileged information exists in machine learning. For example, suppose one wants to predict the outcome of a surgery, whether the patient survives or not. Here, there is much information about the post-surgery symptoms of previous patients. Of course, for predicting the outcome of the current patient's surgery, such information is not available. The available information about previous patients after surgery could be used as privileged information [?].

The Support Vector Machine (SVM) minimizes the following objective function:

$$\min_{f} \sum_{i=1}^{n} \xi_i + \frac{C}{2}\|f\|^2 \tag{1.59a}$$

$$\text{s.t. } y_i f(x_i) \geq 1 - \xi_i \tag{1.59b}$$

$$\xi_i \geq 0. \tag{1.59c}$$

LUPI assumes that there exists a function f^* that can estimate ξ_i from x_i^* [?]. More precisely:

$$\xi_i = \max\{0, f^*(x_i^*, \alpha^*)\}, \quad \alpha^* \in \Lambda. \tag{1.60}$$

This results in the following objective function [?]:

$$\min_{f} \sum_{i=1}^{n} \max\{0, f^*(x_i^*, \alpha^*)\} + \frac{C}{2}\|f\|^2 + \frac{C^*}{2}\|f^*\|^2 \tag{1.61a}$$

$$\text{s.t. } y_i f(x_i) \geq 1 - \max\{0, f^*(x_i^*, \alpha^*)\} \tag{1.61b}$$

$$\alpha^* \in \Lambda. \tag{1.61c}$$

Interestingly, the authors in [?] proved that using privileged information is related to importance weighting, and they showed that privileged features can be expressed by weights for each training example. Furthermore, they showed that the set of solutions of a weighted support vector machine problem always contains the set of solutions of the model proposed by Vapnik for privileged learning. It is also noteworthy that privileged knowledge is completely related to the concept of distillation in deep neural networks [**?**].

Here, we have a number of samples $D_i, i = 1, 2, \ldots n$, each drawn from a distribution P_i. Each sample D_i consists of a set of data points $D_i = \{x_{ij}, j = 1, 2, \ldots n_{ij}\}$. We assume that all the distributions P_i are drawn from a parent distribution [**?**].

In learning on distributions, we want to learn a function $f(P_i)$ for each distribution P_i. As mentioned earlier, because in practice we do not have access to the distributions themselves and only have access to a number of samples drawn from the distributions, learning on distributions is performed using these samples.

Here, we propose that the teacher provides the learning system with information related to the details of the concept being learned. The teacher can provide the student with a classification that indicates an important concept related to the data points, each sample of which is composed of some of them. However, because we do not have access to the examples in learning on distributions, the teacher cannot do this. Instead, the teacher provides labels for a subset of the data points.

Therefore, as privileged information, we have a number of labels for a subset D_i^l of size n_i^l of the data points drawn from the distribution P_i. In other words, the triple $(\{x_{ij}\}_{j=1}^{n_i}, \{x_{ij}, y_{ij}\}_{j \in D_i^l}, y_i)$ is the privileged knowledge for the distribution P_i. Assume $\mu_g(P_i) = \mathbb{E}_{x \sim P_i} g(P_i, x)$ is the mean of the distribution based on the sample drawn from P_i. Let $\mu_{P_i} = \mathbb{E}_{x \sim P_i} \phi(P_i, \cdot)$. We have:

$$\begin{aligned} \mu_g(P_i) &= \mathbb{E}_{x \sim P_i} g(P_i, x) && (1.62) \\ &= \langle g, \mathbb{E}_{x \sim P_i} \phi(P_i, \cdot) \rangle && (1.63) \\ &= \langle g, \mu_{P_i} \rangle. && (1.64) \end{aligned}$$

When we have a characteristic kernel [**?**], we know that μ_{P_i} contains all the information about the distribution P_i.

We define the following objective function:

$$T(f, g, \mu_g) = \frac{1}{2}\|f\|^2 + \frac{C_g}{2}\|g\|^2 + C \sum_{i=1}^{n} [y_i \mu_g(P_i)]_+ + C_v \sum_{i=1}^{n} \sum_{j \in D_i^l} [1 - y_{ij} g(P_i, x_{ji})]_+ . \quad (1.65)$$

To learn the desired classifier, the following optimization problem must be solved:

$$\begin{aligned} &\min_{f,g} T(f, g, \mu_g) && (1.66a) \\ &\text{s.t. } \; y_i f(P_i) \geq 1 - [y_i \mu_g(P_i)]_+ \quad \forall i \in [1, n]. && (1.66b) \end{aligned}$$

In addition to the challenges of the above problem, the structure of the function g must be determined. Based on the work in [**?**], we assume that

$$g(P_i, x_{ji}) = \sum_{t=1}^{n} \sum_{h=1}^{n_t} \beta_{th} K(P_i, P_t) k(x_{ji}, x_{ht}). \quad (1.67)$$

To estimate $\mu_g(P_i)$, we use a sample drawn from P_i:

$$\hat{\mu}_g(P_i) = \frac{1}{n_i} \left(\sum_{j \in D_i} g(P_i, x_{ji}) \right). \quad (1.68)$$

Therefore, the above optimization problem becomes:

$$\min_{f,g} T(f,g,\hat{\mu}_g) \tag{1.69a}$$
$$\text{s.t. } y_i f(P_i) \geq 1 - [y_i \hat{\mu}_g(P_i)]_+ \quad \forall i \in [1,n]. \tag{1.69b}$$

The following theorem proves that the above optimization problem is equivalent to:

Theorem 1.4. *The optimization problem* (1.69) *is equivalent to:*

$$\min_{\eta,\alpha,\theta} \frac{1}{2}\alpha^T YKY\alpha + \frac{1}{2C_g}\left(A^T Y(\eta+\alpha - C\mathbf{1}) + Z\theta\right)^T L\left(LA^T Y(\eta+\alpha-C\mathbf{1}) + LZ\theta\right) \tag{1.70a}$$
$$s.t.\ \eta,\alpha,\theta \geq 0 \tag{1.70b}$$
$$\alpha^T Y\mathbf{1} = 0 \tag{1.70c}$$
$$\mathbf{1}^T A^T Y(\eta+\alpha) + \mathbf{1}^T Z^T\theta = C\mathbf{1}^T A^T Y\mathbf{1} \tag{1.70d}$$
$$\theta \geq C_v B^T \mathbf{1} \tag{1.70e}$$
$$\eta + \alpha \geq (C+\Delta C)\mathbf{1}. \tag{1.70f}$$

Proof. We know that

$$T(f,g,\hat{\mu}_g) = \frac{1}{2}\|f\|^2 + \frac{C_g}{2}\|g\|^2 + C\sum_{i=1}^{n}[y_i\hat{\mu}_g(P_i)]_+ + C_v\sum_{i=1}^{n}\sum_{j\in D_i^l}[1-y_{ij}g(P_i,x_{ji})]_+ . \tag{1.71}$$

To estimate $\hat{\mu}_g(P_i)$, we use a sample drawn from P_i:

$$\hat{\mu}_g(P_i) = \frac{1}{n_i}\left(\sum_{j\in D_i} g(P_i,x_{ji})\right). \tag{1.72}$$

This is a non-linear optimization problem. Its approximation based on the method in [**?**] consists of minimizing the objective function

$$T(f,g) = \frac{1}{2}\|f\|^2 + \frac{C_g}{2}\|g\|^2 + C\sum_{i=1}^{n}(y_i\hat{\mu}_g(P_i)+\xi_i) + \Delta C\sum_{i=1}^{l}\xi_i + C_v\sum_{i=1}^{n}\sum_{j\in D_i^l}[1-y_{ij}g(P_i,x_{ji})]_+ \tag{1.73}$$

with respect to f, g along with the constraints:

$$\xi_i \geq 0 \tag{1.74}$$
$$y_i\hat{\mu}_g(P_i) + \xi_i \geq 0 \tag{1.75}$$
$$y_i f(P_i) + y_i\hat{\mu}_g(P_i) \geq 1 - \xi_i. \tag{1.76}$$

This yields the desired functions. Assume $\zeta_{ij} = [1-y_{ij}g(P_i,x_{ji})]_+$. Then, the optimization problem becomes minimizing the objective function

$$T(f,g) = \frac{1}{2}\|f\|^2 + \frac{C_g}{2}\|g\|^2 + C\sum_{i=1}^{n}(y_i\hat{\mu}_g(P_i)+\xi_i) + \Delta C\sum_{i=1}^{l}\xi_i + C_v\sum_{i=1}^{n}\sum_{j\in D_i^l}\zeta_i \tag{1.77}$$

subject to the constraints:

$$\xi_i \geq 0, \quad \zeta_i \geq 0 \tag{1.78}$$
$$y_i \hat{\mu}_g(P_i) + \xi_i \geq 0 \tag{1.79}$$
$$y_i f(P_i) + y_i \hat{\mu}_g(P_i) \geq 1 - \xi_i \tag{1.80}$$
$$y_{ji} g(P_i, x_{ji}) \geq 1 - \zeta_{ij}. \tag{1.81}$$

Assume

$$f(P_i) = \sum_{t=1}^{n} \gamma_t K(P_i, P_t) + b \tag{1.82}$$
$$g(P_i, x_{ji}) = \sum_{t=1}^{n} \sum_{h=1}^{n_t} \beta_{th} K(P_i, P_t) k(x_{ji}, x_{ht}) + b_g. \tag{1.83}$$

Let L be a matrix with components $L(x_{ji}, x_{ht}) = K(P_i, P_t)k(x_{ji}, x_{ht})$, and let f be a vector with elements $f(P_i)$. Then we have:

$$g(P_i, x_{ji}) = L(x_{ji}, :)\beta + b_g, \quad f(P_i) = K(P_i, :)\gamma + b, \tag{1.84}$$

where β, g are vectors corresponding to components $\beta_{th}, h \in D_i$ and $g(P_i, x_{ji}), j \in D_i$, respectively. Therefore,

$$\|f\|^2 = \gamma^T K \gamma, \tag{1.85}$$
$$\|g\|^2 = (L(., :)\beta)^T L(., :)\beta = \beta^T L \beta. \tag{1.86}$$

Assume $\mathbf{1}_i(x) = \mathbb{I}(x \in D_i)$ is a vector with zero for all data points of other distributions and one for data points drawn from distribution P_i. Then:

$$\mu_g(P_i) = \frac{1}{n_i} \sum_{j \in D_i} g(P_i, x_{ji}) = \frac{1}{n_i} \mathbf{1}_i^T g. \tag{1.87}$$

Therefore, the above optimization problem becomes minimizing the objective function

$$T(f, g) = \frac{1}{2}\|f\|^2 + \frac{C_g}{2}\|g\|^2 + C \sum_{i=1}^{n} \left(y_i \frac{1}{n_i} \mathbf{1}_i^T g + \xi_i \right) + \Delta C \sum_{i=1}^{l} \xi_i + C_v \sum_{i=1}^{n} \sum_{j \in D_i^l} \zeta_i \tag{1.88}$$

subject to the constraints:

$$\xi_i \geq 0, \quad \zeta_i \geq 0 \tag{1.89}$$
$$y_i \frac{1}{n_i} \mathbf{1}_i^T g + \xi_i \geq 0 \tag{1.90}$$
$$y_i f(P_i) + y_i \frac{1}{n_i} \mathbf{1}_i^T g \geq 1 - \xi_i \tag{1.91}$$
$$y_{ji} g(P_i, x_{ji}) \geq 1 - \zeta_{ij}. \tag{1.92}$$

Furthermore, assume $A(i, :) = \frac{1}{n_i} \mathbf{1}_i^T$ for all data points of the distributions, and $B(i, :) = \mathbf{1}_l^{iT}$, where $\mathbf{1}_l^i(x) = \mathbb{I}(x \in D_i^0)$. Let $Y = \mathrm{diag}(y)$ and $Z = \mathrm{diag}(z)$, with $z_{ij} = 0, j \notin D_i^l$ and $z_{ij} = y_{ij}, j \in D_i^l$. The above optimization problem becomes

$$T(f, g) = \frac{1}{2}\gamma^T K \gamma + \frac{C_g}{2}\beta^T L \beta + C \mathbf{1}^T Y A (L\beta + b_g \mathbf{1}) + \xi + \Delta C \mathbf{1}^T \xi + C_v \mathbf{1}^T B \zeta \tag{1.93}$$

along with the constraints:

$$\xi \geq 0, \quad \zeta \geq 0 \tag{1.94}$$
$$YA(L\beta + b_g\mathbf{1}) + \xi \geq 0 \tag{1.95}$$
$$Y(K\gamma + b) + YA(L\beta + b_g\mathbf{1}) \geq 1 - \xi \tag{1.96}$$
$$Z(L\beta + b_g\mathbf{1}) \geq 1 - \zeta. \tag{1.97}$$

The Lagrangian of this optimization problem is:

$$\begin{aligned}
\mathcal{L}(\gamma, b, \beta, b_g, \xi, \zeta; u, v, \eta, \alpha, \theta) &= T(f, g) - u^T\xi - v^T\zeta - \eta^T(YA(L\beta + b_g\mathbf{1}) + \xi) && (1.98)\\
&\quad - \alpha^T(Y(K\gamma + b) + YA(L\beta + b_g\mathbf{1}) + \xi - \mathbf{1}) && (1.99)\\
&\quad - \theta^T(Z(L\beta + b_g\mathbf{1}) + \zeta - \mathbf{1}) && (1.100)\\
&= \frac{1}{2}\gamma^T K\gamma - \alpha^T YK\gamma && (1.101)\\
&\quad + \frac{C_g}{2}\beta^T L\beta - \Big(LA^TY(\eta + \alpha - C\mathbf{1}) + LZ\theta\Big)^T\beta && (1.102)\\
&\quad - \alpha^T Y\mathbf{1}b - \Big(\eta^T YA + \alpha^T YA + \theta^T Z - C\mathbf{1}^T YA\Big)\mathbf{1}b_g && (1.103)\\
&\quad - (v + \theta - C_v B^T\mathbf{1})^T\zeta - (u + \eta + \alpha - (C + \Delta C)\mathbf{1})^T\xi. && (1.104)
\end{aligned}$$

By minimizing with respect to the primal variables, the dual problem is obtained:

$$\frac{\partial\mathcal{L}}{\partial\zeta} = C_v B^T\mathbf{1} - v - \theta = 0 \tag{1.105}$$
$$\frac{\partial\mathcal{L}}{\partial\xi} = (C + \Delta C)\mathbf{1} - u - \eta - \alpha = 0 \tag{1.106}$$
$$\frac{\partial\mathcal{L}}{\partial b_g} = C\mathbf{1}^T YA\mathbf{1} - \eta^T YA\mathbf{1} - \alpha^T YA\mathbf{1} - \theta^T Z\mathbf{1} = 0 \tag{1.107}$$
$$\frac{\partial\mathcal{L}}{\partial\beta} = C_g L\beta + CLA^TY\mathbf{1} - LA^TY\eta - LA^TY\alpha - LZ^T\theta = 0 \tag{1.108}$$
$$\frac{\partial\mathcal{L}}{\partial b} = \alpha^T Y\mathbf{1} = 0 \tag{1.109}$$
$$\frac{\partial\mathcal{L}}{\partial\gamma} = K\gamma - KY\alpha = 0. \tag{1.110}$$

Assuming both K^{-1} and L^{-1} exist, we get:

$$v + \theta = C_v B^T\mathbf{1} \tag{1.111}$$
$$u + \eta + \alpha = (C + \Delta C)\mathbf{1} \tag{1.112}$$
$$\mathbf{1}^T A^T Y(\eta + \alpha) + \mathbf{1}^T Z\theta = C\mathbf{1}^T YA^T\mathbf{1} \tag{1.113}$$
$$C_g\beta = A^T Y(\eta + \alpha - C\mathbf{1}) + Z\theta \tag{1.114}$$
$$\alpha^T Y\mathbf{1} = 0 \tag{1.115}$$
$$\gamma = Y\alpha. \tag{1.116}$$

Therefore, the dual objective function is:

$$\min_{\gamma,b,\beta,b_g,\xi,\zeta} \mathcal{L}(\gamma, b, \beta, b_g, \xi, \zeta; u, v, \eta, \alpha, \theta) = -\frac{1}{2}\alpha^T YKY\alpha \tag{1.117}$$

$$-\frac{1}{2C_g}\left(A^T Y(\eta + \alpha - C\mathbf{1}) + Z\theta\right)^T L\left(LA^T Y(\eta + \alpha - C\mathbf{1}) + LZ\right. \tag{1.118}$$

The dual optimization problem is:

$$\max_{u,v,\eta,\alpha,\theta} \quad -\frac{1}{2}\alpha^T YKY\alpha - \frac{1}{2C_g}\left(A^T Y(\eta + \alpha - C\mathbf{1}) + Z\theta\right)^T L\left(LA^T Y(\eta + \alpha - C\mathbf{1}) + LZ\theta\right) \tag{1.119a}$$

$$\text{s.t.} \;\; u, v, \eta, \alpha, \theta \geq 0 \tag{1.119b}$$

$$\alpha^T Y\mathbf{1} = 0 \tag{1.119c}$$

$$(\eta + \alpha)^T YA\mathbf{1} + \theta^T Z\mathbf{1} = C\mathbf{1}^T YA\mathbf{1} \tag{1.119d}$$

$$v + \theta = C_v B^T \mathbf{1} \tag{1.119e}$$

$$u + \eta + \alpha = (C + \Delta C)\mathbf{1}. \tag{1.119f}$$

This problem is equivalent to:

$$\max_{\eta,\alpha,\theta} \quad -\frac{1}{2}\alpha^T YKY\alpha - \frac{1}{2C_g}\left(A^T Y(\eta + \alpha - C\mathbf{1}) + Z\theta\right)^T L\left(LA^T Y(\eta + \alpha - C\mathbf{1}) + LZ\theta\right) \tag{1.120a}$$

$$\text{s.t.} \;\; \eta, \alpha, \theta \geq 0 \tag{1.120b}$$

$$\alpha^T Y\mathbf{1} = 0 \tag{1.120c}$$

$$\mathbf{1}^T A^T Y(\eta + \alpha) + \mathbf{1}^T Z^T \theta = C\mathbf{1}^T A^T Y\mathbf{1} \tag{1.120d}$$

$$\theta \geq C_v B^T \mathbf{1} \tag{1.120e}$$

$$\eta + \alpha \geq (C + \Delta C)\mathbf{1}. \tag{1.120f}$$

Due to the complexity of this method, it is better to use the much simpler boundary-near approach, which reduces the version space, for active learning. By solving the above problem, we have the two functions $g(P_i, x_{ij})$ and $f(P_i)$. For active learning to label data points in each sampled set, we have:

$$(i, j) = \arg\min_{i,j} |g(P_i, x_{ij})| \tag{1.121}$$

and to query the labels of the distributions, we use the following objective function:

$$P_q = \arg\min_{P_i} |f(P_i)|. \tag{1.122}$$

1.0.1.4 Probabilistic Minimax Active Learning (PMAL)

Introduction In active learning on distributions, we do not have direct access to the distributions but rather to each distribution through a sample drawn from it. Therefore, similar to robust learning, any estimate of an example is inaccurate. This challenge of inaccurate examples adds to the already challenging problem of active learning. To solve these challenges, we provide an upper bound for the risk of the classifier in the next stage of active learning, where more labeled examples become available. Based on this upper bound, to deal with the challenges of robust active learning, we present the Probabilistic Minimax Active Learning (PMAL) method and prove that this algorithm selects an example whose label information minimizes the risk. Unfortunately, computing this objective function for the robust active learning problem is intractable. Therefore, we provide an efficient estimate of the objective function. To further exploit the available information about the distribution estimates, we present a robust active learning method based on Bayesian learning of kernel embeddings of distributions.

In learning on distributions, examples have inherent uncertainty. Each example is estimated using a finite-sized sample. It is well known that estimating a distribution from a finite sample has uncertainty. Therefore, we need a method that is robust against inaccuracies in the examples. Robustness in active learning is much more challenging because another source of uncertainty is added, namely the unknown labels of many points.

To solve this problem with a principled approach, we first consider the active learning problem in its most general form. To this end, we provide an upper bound on the risk of the classifier in the next stage of active learning, where the queried data is added to the labeled data. This risk upper bound is provided before determining the labels of the queried data. Then, examples are queried whose selection minimizes this upper bound. We call the resulting method Probabilistic Minimax Active Learning (PMAL). PMAL can be used in a Bayesian framework.

Table 1.1: Notation table for Probabilistic Minimax Active Learning

Symbol	Description
(X, Y)	Random variables in $\mathbb{R}^d \times \{-1, 1\}$
n	Number of examples
l	Number of labeled examples
$L = \{1, ..., l\}$	Set of indices of labeled data
$L_q = \{1, ..., l, q\}$	Set of indices of labeled data plus query
$U = \{l+1, ..., n\}$	Set of indices of unlabeled data
$D = \{(x_1, y_1), ..., (x_n, y_n)\}$	Set of labeled and unlabeled data
$D_l = \{(x_i, y_i) \mid i \in L\}$	Set of labeled data
$D_u = \{(x_i, y_i) \mid i \in U\}$	Set of unlabeled data
$x_L = (x_1, ..., x_l)$	Ordered list of labeled data
$y_L = (y_1, ..., y_l)$	Their labels
$x_U = (x_{l+1}, ..., x_n)$	Ordered list of unlabeled data
$y_U = (y_{l+1}, ..., y_n)$	Their unknown labels
$x = x_{LU} = (x_1, ..., x_n)$	Ordered list of examples
$y = y_{LU} = (y_1, ..., y_n)$	Their labels

Proposed Method We denote examples in lowercase letters, and bold lowercase letters represent ordered lists, e.g., $\mathbf{x} = (x_1, \ldots, x_n)$ or their labels $\mathbf{y} = (y_1, \ldots, y_n)$. For brevity, the notations are summarized in Table 1.1.

As previously mentioned, there is no accurate estimate of the distributions. Such conditions are similar to robust learning, but also differ in that we have specific information about the estimates of the distributions using samples, which we can exploit.

To effectively exploit this information, we first introduce the Probabilistic Minimax Active Learning (PMAL) approach in Section 1.0.1.4. Using PMAL, we can address uncertainty in active learning, i.e., robust active learning, in a principled manner. Although in the following sections we do not address active learning in the input space, in 1.2.3 we present an algorithm for robust active learning in the input space. This problem provides the reader with the foundations of robust active learning on distributions, so that one can easily understand the difference between the two. Furthermore, in **??**, we present a simpler form of robust active learning on distributions. In 1.2.4, we address the main goal of this chapter, which is robust active learning on distributions. In the following, we first review the definitions and prerequisites for Section 1.2.3.

Background and Definitions

Definition 1.1 (Bayes Decision Function and Bayes Risk)**.** *Assume $\eta^*(x, y)$ is the (unknown) label distribution function given the examples, i.e.,*

$$\eta^*(x, y) = P(Y = y|X = x). \tag{1.123}$$

The optimal classifier or Bayes optimal decision function $g^*(x)$ and the Bayes risk L^* are respectively:

$$g^*(x) = \arg\max_{y\in\mathcal{C}}\{\eta^*(x, y)\}, \quad L^* = P(g^*(X) \neq Y). \tag{1.124}$$

Definition 1.2 (Plug-in Classifier and its Risk)**.** *Let y be the label of point x. Consider the classifier function $g_D(x)$, which we also call the plug-in classifier using the set D. The risk of this classifier is $L_D = P\{g_D(X) \neq Y|D\}$.*

It is clear that

$$\begin{aligned} L_D &= 1 - P\{g_D(X) = Y|D\} && (1.125)\\ &= 1 - \sum_{y\in\mathcal{C}} P\{g_D(X) = y, Y = y|D\} && (1.126)\\ &= 1 - \sum_{y\in\mathcal{C}} \int_{\mathbb{R}^d} \mathbb{I}[g_D(x) = y] P\{Y = y|D, X = x\}\mu(dx). && (1.127) \end{aligned}$$

We define the quantity $P\{Y = y|D, X = x\}$ as η_D, equal to

$$\eta(x, y; \mathbf{x}_D, \mathbf{y}_D) \equiv \eta_D(x, y) := P\{Y = y|D, X = x\}. \tag{1.128}$$

Also, let

$$A(x, D) := \max_{y\in\mathcal{C}}\{\eta^*(x, y) - \eta(x, y; \mathbf{x}_D, \mathbf{y}_D)\}. \tag{1.129}$$

To minimize risk, the plug-in classifier g_D is equal to $g_D(x) = \arg\max_{y\in\mathcal{C}}\{\eta(x, y; \mathbf{x}_D, \mathbf{y}_D)\}$.

To achieve the optimal classifier in the plug-in classifier, the function η must provide a good estimate of the conditional probability of y given x. In other words, to minimize risk, η must be almost equal to the function η^* using data (x, y) as parameters. In fact, it is essential to have some assumption that relates the distribution criterion of the estimated label probability function η to the true label probability function η^*. Without such an assumption, learning is impossible. This essentially means that we relate the data generation process to the label assignment process so that we can optimally use the existence of unlabeled data [**?**]. Tsybakov used the following assumption for this purpose. This assumption holds for various types of estimators and data distributions P [**?**, **?**].

Assumption 1.1. *Assume η_D is an estimator of the function η^* using data D, and $\mathcal{P}$ is a class of probability distributions on (X, Y) such that for constants $C_1 > 0, C_2 > 0$ and a positive sequence a_n, for almost all x with respect to P_X:*

$$\sup_{P \in \mathcal{P}} P\left\{A(x, D) \geq \delta\right\} \leq C_1 \exp(-C_2 a_n \delta^2). \tag{1.130}$$

Lemma 1.1. *Assume that unlabeled data help reduce the risk of the classifier on $\mathcal{P}$. Under Assumption **??**, $\exists c_U > 1$ such that with probability $1 - C_1 \exp(-C_2 a_n \delta^2)$, we have:*

$$\int_{\mathbb{R}^d} A(X, D)\mu(dx) \leq c_U^{-1} \int_{\mathbb{R}^d} A(X, L)\mu(dx) + 2\delta. \tag{1.131}$$

Proof. The risk of the classifiers $g_D(x)$ and $g_L(x)$, $\forall P \in \mathcal{P}$, is:

$$L_D - L^* \leq \int_{\mathbb{R}^d} A(x, D)\mu(dx), \tag{1.132}$$

$$L_L - L^* \leq \int_{\mathbb{R}^d} A(x, L)\mu(dx), \tag{1.133}$$

where $\mu \in \mathcal{P}$. Since we assume that unlabeled data reduce the classifier risk for distribution P, therefore for all $(\mu, \eta) \in \mathcal{P}$:

$$\min_{y_U} \int_{\mathbb{R}^d} A(x, D)\mu(dx) < \int_{\mathbb{R}^d} A(x, L)\mu(dx). \tag{1.134}$$

Based on Assumption **??** and the regularization conditions, we have:

$$\sup_{P \in \mathcal{P}} P\left\{\int_{\mathbb{R}^d} A(x, D)\mu(dx) \leq \delta\right\} \geq 1 - C_1 \exp(-C_2 a_n \delta^2). \tag{1.135}$$

Therefore, with high probability:

$$\int_{\mathbb{R}^d} \max_{y} \min_{y_U} \{\eta^*(x, y) - \eta(x, y; \mathbf{x}, \mathbf{y})\}\mu(dx) \leq \min_{y_U} \int_{\mathbb{R}^d} A(x, D)\mu(dx) + 2\delta. \tag{1.136}$$

Using this relation in the previous one, with probability $1 - C_1 \exp(-C_2 a_n \delta^2)$, we have:

$$\int_{\mathbb{R}^d} A(x, D)\mu(dx) < \int_{\mathbb{R}^d} A(x, L)\mu(dx) + 2\delta. \tag{1.137}$$

Probabilistic Minimax Active Learning At each stage of active learning, we want to select examples whose label knowledge helps build an accurate model. Our wish is that the current model can predict the labels of almost all unlabeled data points. We call these labels the best labels for the current model. However, the worst-case scenario is when, assuming the current model, the probability of the reverse label for an example is high. Here, the reverse label means a label opposite to the one predicted by the current model.

Due to the lack of information about the label of an unlabeled example until querying the user, we must be prepared for the worst-case label for the query point. At the same time, we hope that some unlabeled examples have the best label, i.e., the label predicted by the model. Note that we prefer a simple model to a more complex one. Therefore, for unlabeled data except for the point to be queried, we expect the best label to occur because we assume that the current model is simpler than the models in subsequent stages of active learning, where the number of labeled examples increases. Thus, the motto of this approach is "prepare for the worst and expect the best."

In one stage of active learning, only the subset D_L is used for training, and the labels of unlabeled data $y_i, i \in U$ are unknown. The question is, even if we don't know the label of an example, what will happen to classification accuracy if we add it to the labeled data? At each stage of active learning, only a subset D_L is used for training. It is clear that the labels of unlabeled data $y_i, i \in U$ are unknown.

The fundamental question here is: can we know, before receiving the label of any point, what will happen to the classification accuracy if an example is added to the labeled data?

The theorem we present below answers this question theoretically. Theorem 1.6 provides an un-improvable upper bound for the classifier risk under the above conditions. This means that without having the new labeled data, we cannot know more about the classifier under the stated conditions.

But before presenting this theorem, we need to know more about the risk of the multi-class plug-in classifier. Theorem 1.5 provides an upper bound on the risk for the multi-class classifier. This risk upper bound is based on no additional assumptions about the data distribution.

Theorem 1.5 (Plug-in Risk Bound)**.** *Let $T_D(x, y) = \eta^*(x, y) - \eta_D(x, y)$. Then we have:*

$$L_D - L^* \leq \sum_{i=1}^{|\mathcal{C}|} \mathbb{E}|T_D(X, c_i)| \leq |\mathcal{C}| \max_{y \in \mathcal{C}} \mathbb{E}|T_D(X, y)|. \tag{1.138}$$

Proof. Using the definitions provided in Section 1.2, we have:

$$\eta(X, c) = \mathbb{E}(\mathbb{I}_{\{Y=c\}}|X). \tag{1.139}$$

The risk of the classifier g_D is:

$$L_D = \sum_{y \in \mathcal{C}} \mathbb{E}(\mathbb{I}_{\{g_D(X) \neq y, Y=y\}}) \tag{1.140}$$

$$= \sum_{y \in \mathcal{C}} \sum_{c \in \mathcal{C}-\{y\}} \mathbb{E}(\mathbb{E}(\mathbb{I}_{\{g_D(X)=c\}}\mathbb{I}_{\{Y=y\}}|X)) \tag{1.141}$$

$$= \sum_{y \in \mathcal{C}} \sum_{c \in \mathcal{C}-\{y\}} \mathbb{E}\left(\mathbb{I}_{\{c=\arg\max_z \eta_D(X,z)\}}\eta^*(X, y)\right). \tag{1.142}$$

Similarly, we can easily obtain:

$$L^* = \sum_{y \in \mathcal{C}} \sum_{c \in \mathcal{C}-\{y\}} \mathbb{E}\left(\mathbb{I}_{\{c=\arg\max_z \eta^*(X,z)\}}\eta^*(X, y)\right). \tag{1.143}$$

The set $A_\eta(c) := \{c = \arg\max_z \eta_D(X, z)\}$ can be written as $\cup_{i=1}^{|\mathcal{C}|} B(c, c_i)$, where B is defined using η^*:

$$B(c, c') = \{c = \arg\max_z \eta_D(X, z), c' = \arg\max_z \eta^*(X, z)\}. \tag{1.144}$$

Similarly, the set $A_{\eta^*}(c) := \{c = \arg\max_z \eta^*(X, z)\}$ can be written as $\cup_{i=1}^{|\mathcal{C}|} B(c_i, c)$. Using these sets, we have:

$$\mathbb{E}\{\mathbb{I}_{A_\eta(c)}\eta^*(X, y)\} = \sum_{t \in \mathcal{C}} \mathbb{E}(\mathbb{I}_{B(c,t)}\eta^*(X, y)), \tag{1.145}$$

$$\mathbb{E}\{\mathbb{I}_{A_{\eta^*}(c)}\eta^*(X, y)\} = \sum_{t \in \mathcal{C}} \mathbb{E}(\mathbb{I}_{B(t,c)}\eta^*(X, y)). \tag{1.146}$$

To compute $L_D - L^*$, by canceling equal terms in the following expression, we get:

$$\sum_{c\in\mathcal{C}-\{y\}} \Big(\mathbb{E}\{\mathbb{I}_{A_\eta(c)}\eta^*(X,y)\} - \mathbb{E}\{\mathbb{I}_{A_{\eta^*}(c)}\eta^*(X,y)\}\Big) \tag{1.147}$$

$$= \sum_{c\in\mathcal{C}-\{y\}} \Big(\mathbb{E}\{\mathbb{I}_{B(c,y)}\eta^*(X,y)\} - \mathbb{E}\{\mathbb{I}_{B(y,c)}\eta^*(X,y)\}\Big). \tag{1.148}$$

Using the previous relations, it is obtained that

$$L_D - L^* = \sum_{i=1}^{|\mathcal{C}|-1}\sum_{j=i+1}^{|\mathcal{C}|} \Big(\mathbb{E}\left(\mathbb{I}_{B(c_j,c_i)}(\eta^*(X,c_i) - \eta^*(X,c_j))\right) \tag{1.149}$$

$$+\mathbb{E}\left(\mathbb{I}_{B(c_i,c_j)}(\eta^*(X,c_j) - \eta^*(X,c_i))\right)\Big). \tag{1.150}$$

In $B(c_j,c_i)$, we have $\eta(X,c_j) - \eta(X,c_i) \geq 0$. Similarly, in $B(c_i,c_j)$, we have $\eta(X,c_i) - \eta(X,c_j) \geq 0$. Adding these terms to the right-hand side of the above relation and rearranging terms, we get:

$$L_D - L^* \leq \sum_{i=1,j=i+1}^{|\mathcal{C}|-1,|\mathcal{C}|} \mathbb{E}\left(\mathbb{I}_{E(c_j,c_i)}(|T_D(X,c_i)| + |T_D(X,c_j)|)\right) \tag{1.151}$$

$$\leq \sum_{i=1}^{|\mathcal{C}|} \left(\mathbb{E}(|T_D(X,c_i)|)\right). \tag{1.152}$$

Now that we have proven this theorem, we can present the following theorem for the upper bound of the plug-in classifier assuming any label for the query data. This theorem provides an upper bound for the risk of the plug-in classifier trained on labeled data plus any label on unlabeled data:

Theorem 1.6 (Risk Upper Bound for Active Learning)**.** *If the assumptions of Lemma* **??** *hold, let*

$$\Delta_L \equiv \zeta \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}}\{\eta^*(x,y) - \eta(x,y;\mathbf{x},\mathbf{y})\}\mu(dx). \tag{1.153}$$

Let $\beta = 1 - c_U^{-1}$. Then with probability $1 - C_1\exp(-C_2 a_n \delta^2)$, the upper bound of the risk of the classifier trained using labeled data L and query Q, L_{LQ}, is

$$L_{LQ} - L^* \leq \beta\Delta_L + 2\zeta\delta + \zeta\alpha\left(1 - c_0 \min_{y_Q}\max_{y_U} P(y|x)\right)E_\eta, \tag{1.154}$$

where

$$E_\eta = \mathbb{E}\left(\max_{y\in\mathcal{C}} \eta(X,y;\mathbf{x}_L,\mathbf{y}_L)\right), \tag{1.155}$$

$c_0 > 1$, $\alpha > 0$, and $\zeta \geq 2$ are constants independent of the data.

Proof. Assume that g is the classifier trained using data $\mathbf{x},\mathbf{y}$. Using Theorem 1.5, and with $\|a\| = \max\{a,-a\}$, we have:

$$L_D - L^* \leq |\mathcal{C}| \max_{y\in\mathcal{C}} \mathbb{E}|T_D(X,y)| \tag{1.156}$$

$$= \int_{\mathbb{R}^d} |\mathcal{C}| \max_{y\in\mathcal{C}} \max\{T_D(x,y), -T_D(x,y)\}\mu(dx). \tag{1.157}$$

Then, because

$$\eta^*(x,y) = 1 - \sum_{c\in\{\mathcal{C}-\{y\}\}} \eta^*(x,c), \tag{1.158}$$

$$\eta_D(x,y) = 1 - \sum_{c\in\{\mathcal{C}-\{y\}\}} \eta_D(x,c), \tag{1.159}$$

we have:

$$\max_{y\in\mathcal{C}}\{-T_D(x,y)\} = \max_{y\in\mathcal{C}} \sum_{c\in\{\mathcal{C}-\{y\}\}} (T_D(x,c)) \leq |\mathcal{C}| \max_{y\in\mathcal{C}}\{T_D(x,y)\} = |\mathcal{C}|A(x,D). \tag{1.160}$$

Substituting this into the expression for $L_D - L^*$ leads to the conclusion that $\exists\zeta \geq 2$ such that

$$L_D - L^* \leq \zeta \int_{\mathbb{R}^d} A(x,D)\mu(dx) \equiv \Delta_L. \tag{1.161}$$

Assuming that unlabeled data y_U do not increase the classifier risk for the distribution $(\mu,\eta) \in \mathcal{P}$, therefore minimizing the risk upper bound over y_U results in:

$$L_D - L^* \leq \zeta \min_{y_U} \int_{\mathbb{R}^d} A(x,D)\mu(dx) \tag{1.162}$$

$$= \zeta \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}}\{\eta^*(x,y) - \max_{y_U} \eta(x,y;\mathbf{x},\mathbf{y})\}\mu(dx), \tag{1.163}$$

where the last equality is because the quantity $\eta^*(x,y) - \eta(x,y;\mathbf{x},\mathbf{y})$ is concave with respect to y and convex with respect to y_U. Because y_Q is still unknown, we need to consider the risk in the worst-case scenario. The worst-case scenario is when the label of the queried data maximizes the current estimate of the risk. The upper bound of the risk using labeled data, unlabeled data, and queried examples, based on the previous relation, is:

$$L_D - L^* \leq \zeta \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}}\{\eta^*(x,y) - \max_{y_U} \eta(x,y;\mathbf{x},\mathbf{y})\}\mu(dx) \tag{1.164}$$

$$\leq \zeta \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}}\{\eta^*(x,y) - \min_{y_Q}\max_{y_U} \eta_D(x,y)\}\mu(dx). \tag{1.165}$$

Now, we relate this risk upper bound to the risk of the classifier trained only on labeled data. Let $\beta = 1 - c_U^{-1}$. Using Lemma **??**, it follows that $\exists\alpha > 0$ such that with probability $1 - C_1\exp(-C_2 a_n \delta^2)$, we have:

$$L_D - L^* \leq \zeta \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}} \left\{\eta^*(x,y) - \eta(x,y;\mathbf{x}_L,\mathbf{y}_L) + \eta(x,y;\mathbf{x}_L,\mathbf{y}_L) - \min_{y_Q}\max_{y_U} \eta(x,y;\mathbf{x},\mathbf{y})\right\}\mu(dx) \tag{1.166}$$

$$\leq \zeta\Delta_L + \zeta\alpha \int_{\mathbb{R}^d} \max_{y\in\mathcal{C}} \left\{\beta\eta(x,y;\mathbf{x}_L,\mathbf{y}_L) - \min_{y_Q}\max_{y_U} \eta(x,y;\mathbf{x},\mathbf{y})\right\}\mu(dx) + 2\zeta\delta. \tag{1.167}$$

By definition, we have:

$$\eta(x,y;\mathbf{x},\mathbf{y}) = \frac{P(y,\mathbf{y}|x,\mathbf{x})}{P(y)} \tag{1.168}$$

$$= \frac{P(y,\mathbf{y}_U|x,\mathbf{x},\mathbf{y}_{LQ})P(\mathbf{y}_{LQ})}{P(y)} \tag{1.169}$$

$$= \frac{P(y|x,\mathbf{x},\mathbf{y}_{LQ})P(\mathbf{y}_U|x,y,\mathbf{x},\mathbf{y}_{LQ})P(\mathbf{y}_{LQ})}{P(y)}. \tag{1.170}$$

Assuming independence of y from x_U and also independence of y_U from y, we have:

$$\eta(x, y; \mathbf{x}, \mathbf{y}) = \frac{P(y|x, \mathbf{x}_{LQ}, \mathbf{y}_{LQ}) P(\mathbf{y}_U|x, \mathbf{x}, \mathbf{y}_{LQ}) P(\mathbf{y}_{LQ})}{P(y)} \tag{1.171}$$

$$= \frac{P(y|x, \mathbf{x}_{LQ}, \mathbf{y}_{LQ}) P(\mathbf{y}_U, \mathbf{y}_{LQ}|x, \mathbf{x})}{P(y)}. \tag{1.172}$$

Assume $(\mathbf{y}_U, \mathbf{y}_{LQ}), \mathbf{x}$ are independent of $\mathbf{x}$, we have:

$$\eta(x, y; \mathbf{x}, \mathbf{y}) = \frac{P(y|x, \mathbf{x}_{LQ}, \mathbf{y}_{LQ}) P(\mathbf{y}_U, \mathbf{y}_{LQ}|x)}{P(y)} \tag{1.173}$$

$$= \frac{\eta(x, y; \mathbf{x}_{LQ}, \mathbf{y}_{LQ}) P(y|x)}{P(y)}. \tag{1.174}$$

Because $0 < P(y) < 1$, we can assume that $\exists c_0 > 1$ such that

$$\eta(x, y; \mathbf{x}, \mathbf{y}) \geq c_0 \eta(x, y; \mathbf{x}_{LQ}, \mathbf{y}_{LQ}) P(y|x). \tag{1.175}$$

Using this in the previous relation, with probability $1 - C_1 \exp(-C_2 a_n \delta^2)$, we have:

$$L_{LQ} - L^* \leq \beta \Delta_L + 2\zeta\delta \tag{1.176}$$

$$+ \zeta\alpha \int_{\mathbb{R}^d} \max_{y \in \mathcal{C}} \left\{ 1 - c_0 \min_{y_Q} \max_{y_U} P(y|x) \right\} \eta(x, y; \mathbf{x}_L, \mathbf{y}_L) \mu(dx) \tag{1.177}$$

$$\leq \beta \Delta_L + 2\zeta\delta + \zeta\alpha \left(1 - c_0 \min_{y_Q} \max_{y_U} P(y|x) \right) E_\eta. \tag{1.178}$$

Now the question is: which examples' labels yield the most improvement in classifier accuracy? Based on the above theorem, we can say the examples that minimize the expected risk upper bound for the classifier trained using labeled data plus the queried data.

Corollary 1.1. *Based on Theorem 1.6, the best query for a label that minimizes the expected risk is:*

$$Q = \arg\max_{\{Q, |Q| = b\}} \min_{y_Q} \max_{y_U} P(y|x). \tag{1.179}$$

This theorem essentially states that to achieve higher accuracy, we need to label examples that, regardless of the classifier, provide the most improvement in our knowledge of the conditional probability of labels given examples $P(Y|X)$. If we know this distribution better, building a learner for it is easier. We know that $\forall y_U, \forall y_Q$,

$$\min_{y_Q} P(y|x) \leq \min_{y_Q} \max_{y_U} P(y|x) \leq \max_{y_U} P(y|x). \tag{1.180}$$

At the same time, we know that even using all available knowledge, there are still examples whose label probability is low. This happens at points where we expect misclassification to occur [**?**]. This leads us to the objective function. In this case, to achieve higher accuracy, we should focus on examples/regions where the reverse label, using all available knowledge, is highest. Comparing the objective function with the above relation clarifies that it divides the selected unlabeled data points into two parts.

Our main goal is to provide a method for active learning on distributional data. As we will see later, access to the distribution P_i is through the sample $\{x_{1i}, x_{2i}, \ldots, x_{ni}\}$.

Therefore, we do not have an accurate estimate of P_i. To develop a method for active learning on distributions, this inaccuracy must be used when selecting points. To this end, in the next section, we first present an active learning method in the input space with measurement error.

If the batch size b is small, we can easily solve the objective function. However, if this size is large, approximations such as the one presented in [?] or heuristic techniques can be used.

1.1 Functional Approach to Probabilistic Minimax Active Learning

In this section, we presented a general and theoretical active learning approach. This approach is particularly useful when there is uncertainty in the examples along with uncertainty about the unknown labels of the majority of examples. When examples are inaccurate, it was shown that considering this uncertainty during sample selection in active learning is very effective.

1.1.1 Functional Approach to Probabilistic Minimax Active Learning

Active learning uses sample selection for querying their labels from unlabeled data for learning. By selecting samples, active learning methods try to build a classifier with the minimum labeling cost. At the beginning of the active learning process, the available knowledge is very limited.

Active learning assumes that the contribution of each data point to this limited knowledge is not the same across all data regions. Selecting data points in regions with more knowledge is much more beneficial because these points significantly reduce the labeling cost [?, ?]. The goal of point selection is to select points with the most information and query their labels. These points contain information for the classifier and represent other points. As a criterion for being informative or representative, a classifier is usually trained on the unlabeled data. The classifier estimate is usually done parametrically in algorithms like SVM [?, ?]. These approaches ignore the lack of knowledge in active learning.

When the amount of knowledge is limited, it is more natural to find a function that roughly estimates the learner function. In fact, in such situations, humans usually try to build a rule of thumb and then, by asking simple questions, check whether this rule of thumb is accurate enough in the worst case. A rule of thumb can be considered a simple rule or function that roughly estimates the available data.

One might ask how to choose a rule of thumb in a learning problem, e.g., classification, especially when part of the label data is unknown. The straightforward answer is to choose a rule that works well for the worst-case labels under the current model and then make a query to check its accuracy. This is the basis of the proposed active learning strategy. The proposed approach does not depend on a parametric model but is based on a space of simple rules or functions. In other words, learning is done in function space. Unlike parametric methods, the functional approach formulates the optimization problem in the space of functions, not in the space of model parameters. The goal of the current method is to develop a functional approach to active learning that can consider both the criteria of being informative and being representative.

Working in this function space can be challenging. One way to deal with these challenges is to use functional gradient methods. Similar to gradient-based optimization of an objective function, functional gradient methods move in the direction of the gradient of the objective function. But unlike those methods, where the gradient is a vector in $\mathbb{R}^n$ of model parameters, here the gradient vector is a function. This approach is called functional gradient [?].

Now the question is: how to develop an active learning method with the properties of being informative and representative in function space? The goal of the current chapter is to propose a method to answer this question. To the best of our knowledge, no functional gradient-based method that considers both the criteria of being informative and representative has been proposed so far. The functional gradient approach is deeply related to Ensemble and Boosting methods in the machine learning literature [?]. All Ensemble and Boosting-based methods in active learning have only focused on having a diverse set of classifiers to achieve the informativeness criterion in active learning [?]. Although boosting-based active learning methods have been proposed before [?], these methods do not consider the representativeness of other points in point selection.

When the number of labeled points is very small, both criteria of being informative and representative can improve the current estimate of the classifier. The challenge, of course, is how to consider both criteria and maintain the proper balance between them in the functional approach.

The current chapter proposes a functional gradient approach to probabilistic minimax active learning. In the previous chapter, we proved that PMAL minimizes the classifier risk in the next stage of active learning. Having a meaningful objective function, PMAL balances the two main criteria in point selection. More importantly, PMAL unifies both criteria in the conditional probability of labels given data points. From another perspective, at each stage of active learning, PMAL divides the unlabeled data into two halves using the conditional probability of labels given points.

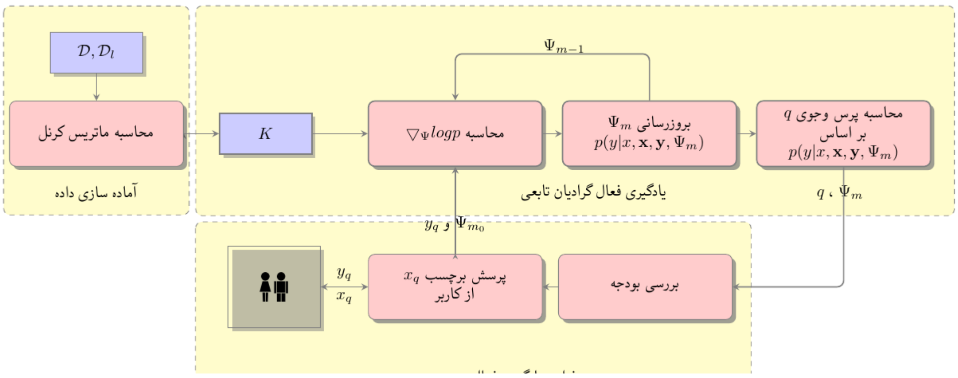


Figure 1.2: Functional gradient approach to Probabilistic Minimax Active Learning

The current chapter introduces a general form and a specific form of the algorithm on a simple class of functions. The general algorithm can be adapted to the specific needs of a domain of applications, while the specific algorithm is a simple form of the algorithm that is easily usable in a wide range of applications. Interestingly, the proposed specific form of the algorithm is inherently related to spectral filtering.

The proposed approach connects active learning to two important research fields: spectral filtering and functional gradient approach to learning. This connection can be explored to provide other active learning algorithms. The proposed functional gradient framework provides a platform for developing better active learning algorithms designed for different applications. The proposed approach is effective in terms of accuracy and speed in general active learning problems. Detailed experiments on 28 datasets show that the proposed functional gradient approach is efficient in selecting points in various applications.

1.1.2 Functional Approach to Active Learning

Assume $\mathbf{y} = [y_1, y_2, \ldots, y_n]^T$ and $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$. In one stage of active learning, we have a set of labeled data $D_L = (\mathbf{x}_L, \mathbf{y}_L)$ and a set of unlabeled data $D_U = (\mathbf{x}_U, \mathbf{y}_U)$. Without loss of generality, assume $\mathbf{y} = (\mathbf{y}_L, \mathbf{y}_U)$ and $\mathbf{x} = (\mathbf{x}_L, \mathbf{x}_U)$, where $\mathbf{y}_U$ is unknown.

Instead of using a parametric model, in this chapter we use a functional approach. We assume the probability function is $P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)$, where Ψ is a function. In each iteration, we estimate Ψ using functional gradient. The function Ψ_m, which is an estimate of Ψ at iteration m, can be estimated by summing a set of gradient steps in the optimal direction:

$$\Psi_m = \Psi_0 + \sum_{t=1}^{m} g_t. \tag{1.181}$$

Similar to gradient in parameter space, each $g_t, t \in [1, m]$ is a functional gradient step, equal to:

$$g_m^* = \mathbb{E}_{x,y}[\nabla_\Psi \log P(y|x, \mathbf{y}, \mathbf{x}, \Psi)|_{\Psi_{m-1}}]. \tag{1.182}$$

Fortunately, the functional gradient at data points can be computed as:

$$g_m(y_i, x_i) = \nabla_\Psi \log P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)|_{\Psi_{m-1}}. \tag{1.183}$$

1.1.3 Functional Gradient Approach and Unlabeled Data

With unlabeled data, we need to be prepared for any label for the unlabeled data. Therefore, to estimate the gradient step, we choose a function from the function class space that has the minimum distance to the gradient at data points with the worst label for the unlabeled data:

$$\{a_m^*, \beta_m\} = \arg\min_{a,\beta} \max_{y_i, i \in D_u} \sum_{i=1}^{n} [g_m(x_i, y_i) - \beta h(x_i; a)]^2. \tag{1.184}$$

If solving this optimization problem is difficult, we can minimize the objective function on the labeled data plus one unlabeled data point j. With this strategy, a simple function is added at each iteration of the algorithm. In other words, at each iteration, we compute the following expression:

$$\{a_m^*, \beta_m\} = \arg\min_{a,\beta} \max_{y_j \in \{-1,1\}} \sum_{i \in D_l \cup \{(x_j, y_j)\}} [g_m(x_i, y_i) - \beta h(x_i; a)]^2. \tag{1.185}$$

In some cases, we might remove an example from the unlabeled data to make the result more dependent on the unlabeled data. At each iteration m, the function Ψ will be:

$$\Psi_m = \Psi_{m-1} + \beta_m^* h_m, \tag{1.186}$$

$$h_m(x_i) = h(x_i; a_m^*). \tag{1.187}$$

The complete algorithm is shown in Algorithm 1. As seen in steps (3)-(8), the proposed approach computes $P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi_m)$ with the functional approach at each step. Step (10) computes the query. Note that this algorithm is general. Often, we have a specific form for $P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)$ based on the available data. For example, the next section considers a normal distribution for $P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)$. To compute the gradient of the function $P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)$ with respect to Ψ, we can use the conditional probability of labels given examples. In the next section, we present a simple algorithm based on Algorithm 1.

Algorithm 1 General Functional Gradient Probabilistic Active Learning

1: **procedure** GFGP ACTIVE LEARNING(K, y_l, Ψ_{m0}, m_l)
2: $\quad m \leftarrow m_0 + 1$
3: $\quad$ **while** $m \leq m_0 + m_l$ or $\|\Psi_m - \Psi_{m-1}\| > \delta$ **do**
4: $\quad\quad g_m(y_i, x_i) = \nabla_\Psi \log p(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi)|_{\Psi_{m-1}}, \forall i \in [1, n]$
5: $\quad\quad$ Compute a_m^*, β_m^* using (3-139) or (3-140).
6: $\quad\quad \Psi_m = \Psi_{m-1} + \beta_m^* h(.,.; a_m^*)$
7: $\quad\quad m \leftarrow m + 1$
8: $\quad$ **end while**
9: $\quad q \leftarrow \arg\max_q \min_{y_q} \max_{y_{\bar{u}}} \prod_{i=1}^n P(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi_m)$
10: $\quad$ **return** $q, \Psi_m(\mathbf{x})$ $\qquad$ ▷ Query instance x_q
11: **end procedure**

1.1.4 Minimax Approach to Active Learning in the L2Boost Model

This section considers the following model for the conditional probability of the label given x:

$$y_i = \Psi_m(x_i, \mathbf{x}, \alpha) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \tag{1.188}$$

where $\mathcal{N}$ is the normal distribution and σ^2 is its variance. Therefore,

$$p(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi) = \mathcal{N}(y_i; \Psi_m(x_i, \mathbf{x}, \alpha), \sigma^2). \tag{1.189}$$

To simplify notation, assume $\Psi(\mathbf{x}, \alpha) = [\Psi_m(x_1, \mathbf{x}, \alpha), \ldots, \Psi_m(x_n, \mathbf{x}, \alpha)]^T$ and $p(\mathbf{y}|\mathbf{x}, \alpha) = \prod_{i=1}^n p(y_i|x_i, \mathbf{y}, \mathbf{x}, \Psi(., \alpha))$. Then:

$$p(\mathbf{y}|\mathbf{x}) = \int p(\mathbf{y}|\mathbf{x}, \alpha) p(\alpha) d\alpha. \tag{1.190}$$

Unfortunately, computing this integral is challenging. Instead, the current study uses the maximum value of $p(\mathbf{y}|\mathbf{x}, \alpha)$ for $p(\mathbf{y}|\mathbf{x})$. To compute the maximum, we use the functional gradient approach. The following lemma provides an approximation for the PMAL objective function for this model.

Lemma 1.2. *Let $K = \mathbf{x}^T \mathbf{x}$. Assuming a linear model*

$$h(\mathbf{x}, \alpha) = K\alpha, \tag{1.191}$$

in the above functional gradient model, we have:

$$p(\mathbf{y}|\mathbf{x}) \propto \exp\left(-\frac{1}{2}\mathbf{y}^T M \mathbf{y}\right), \tag{1.192}$$

where $M = \sum_{t=1}^m R^t$, $R = (I - \eta K)$, and $\eta \in (0, 1/\lambda_{\max}(K))$.

Proof. Based on the functional gradient method:

$$\alpha_t = \alpha_{t-1} - \eta_t \nabla_\Psi \log p(\mathbf{y}|\mathbf{x},\alpha)|_{\alpha_{t-1}} \tag{1.193}$$
$$= \alpha_{t-1} + \eta_t(\mathbf{y} - \Psi_t(\mathbf{x},\alpha_{t-1})). \tag{1.194}$$

Consequently:

$$\alpha_t = \alpha_{t-1} + \eta_t(\mathbf{y} - \Psi_{t-1}(\mathbf{x},\alpha_{t-1}) - K\alpha_{t-1}) \tag{1.195}$$
$$= (I - \eta_t K)\alpha_{t-1} + \eta_t(\mathbf{y} - \Psi_{t-1}(\mathbf{x},\alpha_{t-1})) \tag{1.196}$$
$$= (I - \eta_t K)\{(I - \eta_{t-1}K)\alpha_{t-2} + \eta_{t-1}(\mathbf{y} - \Psi_{t-2}(\mathbf{x},\alpha_{t-2}))\} + \eta_t(\mathbf{y} - \Psi_{t-1}(\mathbf{x},\alpha_{t-1})) \tag{1.197}$$
$$= (I - \eta_t K)(I - \eta_{t-1}K)\alpha_{t-2} + (I - \eta_t K)\eta_{t-1}(\mathbf{y} - \Psi_{t-2}(\mathbf{x},\alpha_{t-2})) + \eta_t(\mathbf{y} - \Psi_{t-1}(\mathbf{x},\alpha_{t-1})) \tag{1.198}$$
$$= \prod_{i=0}^{t-1}(I - \eta_{t-i}K)\alpha_0 + \left\{\sum_{i=0}^{t-1}\eta_{t-i}\prod_{j=0}^{i}(I - \eta_{t-j+1}K)(\mathbf{y} - \Psi_{t-i-1}(\mathbf{x},\alpha_{t-i-1}))\right\}, \tag{1.199}$$

where $\eta_{t+1} = 0$. Using this relation and the expression for h in the integral for $p(\mathbf{y}|\mathbf{x})$, we get:

$$p(\mathbf{y}|\mathbf{x},\alpha_{t+1}) \propto \exp\left(-\frac{1}{2}\|\mathbf{y} - \Psi_t(\mathbf{x},\alpha_t) - K\alpha_t\|^2\right) \tag{1.200}$$
$$\propto \exp\left(-\frac{1}{2}\|\mathbf{y} - \Psi_t(\mathbf{x},\alpha_t) - K\{(I - \eta_t K)\alpha_{t-1} + \eta_t \mathbf{y}\}\|^2\right) \tag{1.201}$$
$$\propto \exp\left(-\frac{1}{2}\|(I - \eta_t K)\mathbf{y} - K(I - \eta_t K)\alpha_{t-1}\|^2\right) \tag{1.202}$$
$$\propto \exp\left(-\frac{1}{2}\left\|\mathbf{y} - K\left\{\prod_{i=0}^{t-1}(I - \eta_{t-i}K)\alpha_0 + \left(\sum_{i=0}^{t-1}\eta_{t-i}\prod_{j=0}^{i}(I - \eta_{t-j+1}K)\right)\mathbf{y}\right\}\right\|^2\right) \tag{1.203}$$
$$\propto \exp\left(-\frac{1}{2}\left\|\left\{I - K\sum_{i=0}^{t-1}\eta_{t-i}\prod_{j=0}^{i}(I - \eta_{t-j+1}K)\right\}\mathbf{y} - K\prod_{i=0}^{t-1}(I - \eta_{t-i}K)\alpha_0\right\|^2\right). \tag{1.204}$$

Starting from zero, $\alpha_0 = 0$, after the t-th iteration:

$$p(\mathbf{y}|\mathbf{x}) \propto \exp\left(-\frac{1}{2}\mathbf{y}^T M \mathbf{y}\right), \tag{1.205}$$

where

$$M = I - K\sum_{i=0}^{t-1}\eta_{t-i}\prod_{j=0}^{i}(I - \eta_{t-j+1}K). \tag{1.206}$$

Assume $\eta_i = \eta, \forall i$, and $R = (I - \eta K)$, then:

$$M = I - \eta K\sum_{i=0}^{t-1}(I - \eta K)^i \tag{1.207}$$
$$= I + (R - I)\sum_{i=0}^{t-1}R^i \tag{1.208}$$
$$= I + \sum_{i=0}^{t-1}R^{i+1} - \sum_{i=0}^{t-1}R^i \tag{1.209}$$
$$= R^t. \tag{1.210}$$

Proposition 1.1. *Let $d_k, k = 1, \ldots, n$ be the eigenvalues of the kernel matrix $K = U \operatorname{diag}(d_k) U^T$, and let $f = [f(x_1), \ldots, f(x_n)]^T$ be a vector containing the unknown values of the function f at the data points. Then the bias and variance are:*

$$b^2(m, K; f) \propto \mu^{2m+2} f^T U \operatorname{diag}(1/(d_k + \mu)) U^T f, \tag{1.211}$$

$$\operatorname{var}(m, K; \sigma^2) \propto \sigma^2 \sum_{t=1}^{n} \left(1 - \frac{\mu^{m+1}}{(d_k + \mu)^{m+1}}\right)^2. \tag{1.212}$$

Proof. Let $K = UDU^T$ be the SVD decomposition of K. We show that the following matrix is symmetric:

$$\begin{aligned} H &= K(K + \mu I)^{-1} K^{\dagger} K & (1.213)\\ &= UDU^T U(D + \mu I)^{-1} U^T U D^{\dagger} U^T U D U^T & (1.214)\\ &= UD(D + \mu I)^{-1} D^{\dagger} D U^T & (1.215)\\ &= KK^{\dagger}(K + \mu I)^{-1} K. & (1.216) \end{aligned}$$

This relation shows the relationship between the eigenvalues of H and K. Let d_k be an eigenvalue of K, then there is a one-to-one relationship between d_k and λ_k:

$$\lambda_k = \frac{d_k}{d_k + \mu} = \frac{1}{1 + \frac{\mu}{d_k}}. \tag{1.217}$$

Proposition 3 in [**?**] calculates the MSE error for the boosting operator:

$$\begin{aligned} \mathrm{MSE} &= \mathrm{bias}^2 + \mathrm{var} & (1.218)\\ \mathrm{bias}^2(m, H; f) &= n^{-1} f^T U \operatorname{diag}((1 - \lambda_k)^{2m+2}) U^T f, & (1.219)\\ \operatorname{var}(m, H; \sigma^2) &= n^{-1} \sigma^2 \sum_{k=1}^{n} \left(1 - (1 - \lambda_k)^{m+1}\right)^2, & (1.220) \end{aligned}$$

where $\operatorname{var}(\epsilon_i) = \sigma^2$, $\mathbb{E}(\epsilon_i) = 0$, $Y_i = f(x_i) + \epsilon_i$, and f are the function values computed at the data points. Relation (3-156) shows that the number of iterations in the boosting algorithm plays the role of a regularization parameter. As it increases, the function becomes more complex. It is important to note that because this matrix is symmetric, all its eigenvalues are real and between zero and one, i.e., $\{\lambda_i \in \mathbb{R} | 0 \leq \lambda_i < 1\}$. Based on Proposition 3 from [**?**], the result is proven.

Let $\bar{U} = U - \{q\}$. With this approximation, the objective function is:

$$\begin{aligned} q &= \arg\max_{q \in U} \min_{y_q} \max_{y_{\bar{U}}} p(y|x) & (1.221)\\ &= \arg\max_{q \in U} \min_{y_q} \max_{y_{\bar{U}}} \log p(y|x) & (1.222)\\ &= \arg\min_{q \in U} \max_{y_q} \min_{y_{\bar{U}}} \mathbf{y}^T M \mathbf{y}. & (1.223) \end{aligned}$$

The resulting algorithm is Algorithm 2. To ensure that matrix M is positive semi-definite and the algorithm is stable, we choose $\eta = 1/\lambda_{\max}(K)$, where $\lambda_{\max}(K)$ is the largest eigenvalue of K.

As previously mentioned, Algorithm 1 works on a function space for active learning. At each iteration of the algorithm, a functional component, which is itself a functional

Algorithm 2 Active Kernel L2Boost Algorithm

1: **procedure** AL2BOOSTKERNEL(K, y_l) ▷ L2Boost Kernel Active Learning algorithm
2: $\quad \eta \leftarrow 1/\lambda_{\max}(K)$, $n_l \leftarrow \text{sizeof}(y_l)$
3: $\quad M, R, L \leftarrow I$
4: $\quad L \leftarrow (I - \eta K)$
5: $\quad$ **for** $t = 1$ to $4 * n_l$ **do** ▷ Compute the matrix L
6: $\quad\quad R \leftarrow RL$
7: $\quad\quad M \leftarrow M + R$
8: $\quad$ **end for**
9: $\quad$ Compute $q = \arg\min_q \max_{y_q} \min_{y_u} \mathbf{y}^T M \mathbf{y}$
10: $\quad$ **return** q ▷ Query instance x_q
11: **end procedure**

gradient of $p(y|\mathbf{x})$ computed in step (4), is added to the function approximation of the classifier. Using the objective functions (3-139) or (3-140), a function of the form $h(.,.;a)$ is chosen that is closest to the computed gradient for the current model in step (4) with the worst-case label for the unlabeled data.

Because the labels of the unlabeled data are unknown, the objective function (3-139) or (3-140) is important for accessing a functional component for active learning. Using these objective functions, a functional component is added to the function Ψ_{m-1} that is suitable for all labels of the unlabeled data that will later be revealed in the active learning process. This loop is repeated until a known number of iterations or a significant change in the functional approximation Ψ_m is observed.

The objective function in line (9) computes the query. This objective function calculates the worst-case label for the unlabeled data $\bar{y}_u$. Therefore, the calculation method is completely consistent with how the components of the function Ψ_m were calculated in previous steps.

Algorithm 2 is much simpler due to the normal assumption for $p(y|\mathbf{x}, \mathbf{x}, \mathbf{y}, \Psi)$ and the linearity of h. At each iteration of the algorithm, a special vector is added to the approximation. The result is an efficient algorithm for selecting examples.

The objective function in Lemma 3.1.2 includes the matrix $M = \sum_{t=1}^{m}(I - \eta K)^t$, which is an admissible spectral filter applied to the matrix K [**?**]. Other admissible spectral filters, such as Iterated Tikhonov $G_\lambda(\sigma) = \frac{(\sigma+\lambda)^t - \lambda^t}{\sigma(\sigma+\lambda)^t}$, exist. With the help of the proposed method, any of these spectral filters can be used in active learning algorithms with different properties.

The proposed approach regularizes the learning function in a different way. As previously mentioned, at each iteration of the algorithm, adding a component to the function is stage-wise. In other words, each new component is added to the function without updating the previously added components. Each component tries to minimize the difference between the desired function and the updated function Ψ_m as much as possible. Therefore, the newer components change faster. Early stopping prevents the algorithm from adding fast-changing components. In other words, it regularizes the way the function Ψ_m is built. This phenomenon is well known in the literature on both functional gradient and spectral filters [**?**, **?**].

Time Analysis The experiments in the case study show that the best number of iterations is a multiple of the number of labeled data. This finding is very understandable because when there are very few labeled examples, adding small components to the resulting approximation function might overfit the classifier. Therefore, in the early stages of active learning, very few iterations are needed to compute the classifier. But when the number of labeled data increases, we should have more iterations to be prepared for the increasing complexity of the classifier function for the data. In fact, in spectral filtering, the number of iterations plays the role of the regularization parameter [**?**]. This phenomenon also holds in functional gradient. Therefore, the number of iterations in Algorithm 2 is $O(n_l)$.

Furthermore, the time spent in each iteration of Algorithm 2 is $O(n^2)$. Computing q involves solving a quadratic objective function and therefore has time complexity $O(n^3)$. Because the kernel matrix is fixed throughout the active learning process, we only need to compute its largest eigenvalue once, and therefore its computation time is negligible. Since the number of iterations is $O(n_l)$, the time complexity of the algorithm is $O(n^3 + n_l n^2)$ or $O(n^3)$ since $n_l < n$.

1.2 Active Learning in the Distribution Domain

If the data of interest are distributions, then learning and active learning on this data will differ significantly from learning on vectors. The reason is that we usually do not have access to the data distribution, but we can sample from the distribution. Therefore, we have sampled a number of samples from each distribution. The accuracy of the estimate of each distribution depends on the number of samples we have selected from it. Consequently, learning based on these samples depends on the number of samples selected from each distribution.

1.2.1 Learning on Distributions

One approach to learning on distributions is to map the distributions to a Hilbert space using the Hilbert Space Embedding of Distributions. This method allows us to view distributions as a point in the Hilbert space. As a result, all common learning methods that can use kernels can be used for learning on distributions.

However, this has problems. One of these problems is the inaccuracy of the estimate of the distributions due to the small number of samples in the transformation domain. Often, we have few samples from some distributions and more samples from others. This causes the accuracy of the samples to differ from each other. Therefore, the sample accuracy is not uniform.

A proposal is to estimate the samples in a weighted manner proportional to their accuracy so that learning is not biased. This allows us to exploit the difference in the estimation accuracy of samples in learning on distributions. In this method, samples do not have the same value. Samples with higher accuracy have more value, and samples with lower accuracy have less value. (This issue can be calculated based on the bounds obtained for the problem. These bounds are discussed in [**?**]. The main paper on the relevant bound is [**?**].)

Active learning algorithms try to reduce the cost of building learning systems by reducing the number of labeled examples needed. In recent years, many learning systems have been developed for higher-level and more abstract forms of data, such as learning on distributions. In learning on distributions, each example is itself a distribution. In this chapter, we propose active learning on distributions as a higher-level and more effective type of active learning algorithm. In active learning on distributions, we do not have direct access to the distributions but rather to each distribution through a sample drawn from it. Therefore, similar to robust learning, any estimate of an example is inaccurate. This challenge of inaccurate examples adds to the already challenging problem of active learning. To solve these challenges, we provide an upper bound for the risk of the classifier in the next stage of active learning, where more labeled examples become available. Based on this upper bound, to deal with the challenges of robust active learning, we present the Probabilistic Minimax Active Learning (PMAL) method and prove that this algorithm selects an example whose label information minimizes the risk. Unfortunately, computing this objective function for the robust active learning problem is intractable. Therefore, we provide an efficient estimate of the objective function. To further exploit the available information about the distribution estimates, we present a robust active learning method based on Bayesian learning of kernel embeddings of distributions. Experiments demonstrate the efficiency of the algorithm on a number of synthetic and real distributional datasets.

To learn a concept, learning systems require knowledge. The goal of learning systems is to predict the labels of test data that were not seen during the construction of the learning system. Without knowledge, learning systems are unable to generalize the concept to unseen data.

Transferring knowledge from human to machine is a fundamental aspect of learning systems. Knowledge is often provided in the form of supervision on data. Recent decades have witnessed the emergence of new paradigms for transferring knowledge from human to machine. One of these paradigms is active learning, where the machine selects data for supervision.

To minimize the need for supervision, active learning deals with the intelligent use of limited supervision resources and human knowledge. By cleverly selecting parts of the data and querying a human user for supervisory data for them, active learning reduces the effort required to achieve a certain level of performance. However, humans usually perceive knowledge in abstract and high-level forms, while algorithms and machines usually require knowledge in low-level forms, such as labels on simple vectors.

In practice, transferring knowledge in high-level forms can be an advantage in active learning because users may be able to provide information better. In such situations, active learning algorithms may obtain richer information from the user compared to receiving labels for simple vectors. For example, [?] presented a generalized query type equivalent to a large number of specific queries for rich communication between the user and the machine.

Since the last decade, learning systems have gradually become more complex and have worked on higher-level data such as structured data or distributional data. In learning from structured data, the data has a structure like a graph [?], while in learning on distributions [?], each data point is itself a distribution. It is highly desirable to provide effective methods to reduce supervision costs on this type of data.

In many applications, it is more natural to assume that knowledge does not reside in each member of the sample, but rather in the distributions from which the sample is drawn. For example, in tasks such as anomaly detection, each point might be normal, but a group of points might be anomalous [?]. In such applications, it is better to model the problem as learning on distributions. Such problems include learning from groups, such as group anomaly detection and preference learning. In recent years, many methods have been developed for learning on distributions [?, ?, ?, ?].

In this chapter, we present an active learning approach on distributions. In learning on distributions, examples have inherent uncertainty. Each distributional example is estimated using a finite-sized sample. It is well known that estimating a distribution from a finite sample has uncertainty. The proposed method is robust against inaccuracies in the examples. Robustness in active learning is much more challenging when we have partial information about the data.

To solve this problem with a principled approach, we first consider the active learning problem in its most general form. In this setting, we provide an upper bound on the risk of the classifier in the next stage of active learning with an increased number of labeled data, under conditions where we have not yet received any new labeled examples. Then, an example is queried in such a way that its selection minimizes this upper bound. We call the resulting method Probabilistic Minimax Active Learning (PMAL). PMAL can be used in a Bayesian framework.

Based on PMAL, we present a robust active learning algorithm. Here, examples can be inaccurate. Unfortunately, computing PMAL for robust active learning is intractable. Therefore, we provide an efficient estimate for Probabilistic Minimax Active Robust Learning (PMARL). The desired estimate is accurate in the sense that there exist labels for the unlabeled data for which the estimation error is negligible.

The learning on distributions problem can be considered as a specific type of robust learning. Although we present a general robust active learning algorithm that considers uncertainty in example estimation, our focus here is on how to deal with uncertainty during the estimation of distributional examples. Note that here, robust learning differs from some other research [?, ?], where the goal is robustness against outlier and noisy data. The paper [?] also mentions this difference, stating that there are two categories of robust learning algorithms. One category is robust against inaccuracies in the example, such as the algorithms proposed in this chapter, and the other category is robust against noisy data, which we addressed in previous chapters. To the best of our knowledge, this is the first active learning algorithm specifically designed for learning on distributions.

Any learning on distributions algorithm is based on distribution estimation. Recently, approaches for distribution estimation in kernel-based methods have been proposed.

Kernel-based approaches developed for learning on distributions usually use the Kernel Mean Embedding of distributions [?], where each distribution is represented as a mean function in a Hilbert space. Because the sample size for a distribution is finite, the Empirical Kernel Mean Embedding is used, which is merely an average of the kernel function for each point in the sample [?]. Recently, it has been shown that the Empirical Kernel Mean Embedding is not the best estimate for distributions [?]. We know well that the Kernel Mean Embedding of distributions has an uncertainty that is a function of the sample size.

Many learning on distributions algorithms ignore the inherent uncertainty of the estimate. Using the recently proposed Bayesian learning of kernel embedding approach [?], in the current chapter, we propose a method for robust active learning on distributions. We call this method Probabilistic Minimax Active Robust Learning on Distributions with Bayesian learning of kernel embeddings. This method uses Bayesian learning of kernel embeddings to deal with uncertainty in the kernel embedding estimate. Experiments show that the proposed method is efficient in estimating inherent uncertainty and selecting distributional examples using samples.

To summarize, two main innovations are presented in this work:

1. A robust active learning algorithm on distributions, in the sense that sample estimates have uncertainty.
2. Probabilistic Minimax Active Learning (PMAL), which is applicable to active learning problems and can be used in Bayesian learning in algorithms. We proved that this method selects examples in a way that minimizes the classifier risk.

Table 1.2: Notation table for active learning on distributions

Symbol	Description
	Learning on Distributions Notation
μ^*_P	Kernel embedding of distribution P
$\hat{\mu}_{P_i}$	Empirical kernel embedding of distribution P_i
$\hat{\mu}_{P_i}(x)$	Evaluation of the kernel embedding value for distribution P_i at point x
$x_i \sim P_i$	Sample drawn from distribution P_i
$\forall u \in [1..n_i], x_{iu} \in \mathbb{R}^d$	$x_i = \{x_{i1}, x_{i2}, \ldots, x_{in_i}\}$
$\hat{\mu}_{P_i}(\mathbf{x}_i) \in \mathbb{R}^{n_i}$	Evaluation of the kernel mean embedding of P_i at each point of $\mathbf{x}_i$
$\hat{\boldsymbol{\mu}}_P = [\hat{\mu}_{P_1}, \hat{\mu}_{P_2}, \ldots, \hat{\mu}_{P_n}]^T$	List of empirical kernel mean embeddings of distributions $P_1, \ldots, P_n$
$\hat{\boldsymbol{\mu}}_P(\mathbf{x})$	Ordered list of evaluations of kernel embeddings from the samples

In the remainder of this chapter, in 1.2.2, we present the proposed approach. The Probabilistic Minimax Active Learning method is presented in 1.0.1.4, and the robust active learning method is presented in 1.2.3. In 1.2.4, the main active learning on distributions method is discussed. After that, the results of experimental evaluations and finally the conclusion are presented.

1.2.2 Active Learning on Distributions based on Probabilistic Minimax Active Learning

We denote examples in lowercase letters, and bold lowercase letters represent ordered lists, e.g., $\mathbf{x} = (x_1, \ldots, x_n)$ or their labels $\mathbf{y} = (y_1, \ldots, y_n)$. For brevity, the notations are summarized in Table 1.2.

As previously mentioned, there is no accurate estimate of the distributions. Such conditions are similar to robust learning, but also differ in that we have specific information about the estimates of the distributions using samples, which we can exploit.

To effectively exploit this information, we first introduce the Probabilistic Minimax Active Learning (PMAL) approach in Section 1.0.1.4. Using PMAL, we can address uncertainty in active learning, i.e., robust active learning, in a principled manner. Although in the following sections we do not address active learning in the input space, in 1.2.3 we present an algorithm for robust active learning in the input space. This problem provides the reader with the foundations of robust active learning on distributions, so that one can easily understand the difference between the two. Furthermore, in **??**, we present a simpler form of robust active learning on distributions. In 1.2.4, we address the main goal of this chapter, which is robust active learning on distributions. In the following, we first review the definitions and prerequisites for Section 1.2.3.

1.2.3 Robust Bayesian Active Learning in the Input Space

In robust learning settings, each example is inaccurate. There can be many reasons for this inaccuracy. In any case, here we assume that there is a probability of error in the examples. On the other hand, the performance of the learning algorithm should not be degraded due to this issue. In other words, the algorithm must be robust against this inaccuracy. Let x_i denote a data point and x_i^* its true value.

Here we assume that each data point x_i has an uncertainty represented by the following conditional distribution:

$$x_i^*|x_i \sim \mathcal{N}(0, \sigma_i^2). \tag{1.224}$$

Assuming a linear model, we have

$$y_i = x_i^{*T} w + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2). \tag{1.225}$$

Therefore, to compute $P(\mathbf{y}|\mathbf{x})$, we first compute $P(y_i|x_i, w)$. We have:

$$\begin{aligned} P(y_i|x_i, w) &= \int P(y_i|w, x_i^*) P(x_i^*|x_i) dx_i^* && (1.226) \\ &= \int \mathcal{N}(y_i; x_i^{*T} w, \sigma^2) \mathcal{N}(x_i^*; x_i, \sigma_i^2) dx_i^* && (1.227) \\ &= \mathcal{N}(y_i; x_i^T w, \sigma^2 + \sigma_i^2 \|w\|^2). && (1.228) \end{aligned}$$

The first equality is the law of total probability. The second equality follows from relation (3-159). The last equality is the well-known integral of two Gaussians. Let $\sigma_i(w) = \sigma^2 + \sigma_i^2\|w\|^2$. Then we have:

$$P(\mathbf{y}|\mathbf{x}, w) \propto \prod_i \sigma_i(w)^{-1/2} \exp\left(-\frac{1}{2}\sum_i \frac{(y_i - w^T x_i)^2}{\sigma_i(w)}\right). \tag{1.229}$$

This relation states that if a point has higher variance compared to other points, it should have less influence on the classifier, and therefore its probability of being selected in active learning should be lower than other points that have more influence on the classifier. This relation also shows that if the classifier is simple, i.e., if $\|w\|$ is small, the variance of the conditional distribution $P(\mathbf{y}|\mathbf{x}, w)$ will be less affected by the uncertainty of a point. Considering the prior knowledge $p(w) = \mathcal{N}(0, \sigma_w^{-2} I)$ for w to compute $p(\mathbf{y}|\mathbf{x})$, we have:

$$P(\mathbf{y}|\mathbf{x}) = \int P(\mathbf{y}|\mathbf{x}, w) P(w) dw. \tag{1.230}$$

Unfortunately, this integral is intractable. More importantly, $\mathbf{y}_U$ is unknown. Therefore, we need to find an estimate of $P(\mathbf{y}|\mathbf{x})$ as a function of $\mathbf{y}_U$. We also need to find a way to compute it efficiently.

To this end, we consider the Gaussian probability density function approximation $\tilde{P}(\mathbf{y}|\mathbf{x}, w)$. This function is equal to:

$$\tilde{P}(\mathbf{y}|\mathbf{x}, w) = \prod_{i=1}^{n} \mathcal{N}(y_i; w^T x_i, \sigma^2 + \sigma_i^2 v^2), \tag{1.231}$$

where v will be determined shortly. Using $\tilde{P}$ instead of P solves the intractability of the integral. But the question is how close these are for each value of $\mathbf{y}_U$, under conditions where we cannot compute the integral?

The theorem below approximates the KL divergence between the two distributions P and $\tilde{P}$. This theorem is based on recent advances in the Laplace approximation of intractable integrals. For more information, see [**?**, **?**, **?**].

Theorem 1.7. *Let*

$$w^* = \arg\max_w P(\mathbf{y}, w|\mathbf{x}), \quad \lambda = \frac{1}{\max_i\{\sigma_i^2\}}. \tag{1.232}$$

Also,

$$h(w; \mathbf{y}, \mathbf{x}) = \frac{1}{2}\sum_{i=1}^{n}\left\{\frac{(y_i - w^T x_i)^2}{\sigma_i(w)} + \log\sigma_i(w)\right\} + \frac{1}{2\sigma_w^2}\|w\|^2, \tag{1.233}$$

$$g(w, v) = \frac{1}{2}\sum_{i=1}^{n}\left\{\frac{\sigma_i^2(\|w\|^2 - v^2)(y_i - w^T x_i)^2}{\sigma_i(v)\sigma_i(w)} + \log\frac{\sigma_i(v)}{\sigma_i(w)}\right\}. \tag{1.234}$$

Then we have:

$$\mathrm{KL}(P(\mathbf{y}, w|\mathbf{x})||\tilde{P}(\mathbf{y}, w|\mathbf{x})) = c_\lambda\left\{\frac{nD(P, \tilde{P}; \mathbf{y}, \mathbf{x}, \lambda, v)}{\sqrt{\det D_h(w^*; \mathbf{y}, \mathbf{x})}} + \epsilon(\lambda)\right\}, \tag{1.235}$$

where

$$D(P, \tilde{P}; \mathbf{y}, \mathbf{x}, \lambda, v) \propto \sqrt{\frac{p}{e}\frac{\det D_h(w^*; \mathbf{y}, \mathbf{x})}{g(w^*, v)}}, \tag{1.236}$$

and $c_\lambda = \left(\frac{2\pi}{\lambda}\right)^{d/2}$, $\epsilon(\lambda) = e^{-h(w^*;\mathbf{y},\mathbf{x})}O(1/\lambda)$ *as* $\lambda \to \infty$.

Proof. The KL divergence between these two distributions is:

$$D_{\tilde{P}}^{P} = \mathrm{KL}(P||\tilde{P}) = \int_{\mathbb{R}^d} P(\mathbf{y}, w|\mathbf{x})\log\frac{P(\mathbf{y}, w|\mathbf{x})}{\tilde{P}(\mathbf{y}, w|\mathbf{x})}dw. \tag{1.237}$$

We know that

$$P(\mathbf{y}, w|\mathbf{x}) = (2\pi)^{-d}\exp(-h(w; \mathbf{y}, \mathbf{x})). \tag{1.238}$$

Also, let

$$\lambda = 1/\max_{i=1}^{n}\sigma_i^2. \tag{1.239}$$

Note that as $\lambda \to \infty$, $P(\mathbf{y}, w|\mathbf{x})$ becomes a univariate Gaussian. For simplicity, let

$$f(w; \mathbf{y}, \mathbf{x}, \lambda) = h(w; \mathbf{y}, \mathbf{x})/\lambda. \tag{1.240}$$

Therefore, we have

$$P(\mathbf{y}, w|\mathbf{x}) = (2\pi)^{-d}\exp(-\lambda f(w; \mathbf{y}, \mathbf{x}, \lambda)). \tag{1.241}$$

We easily see that

$$\log\frac{P(\mathbf{y}, w|\mathbf{x})}{\tilde{P}(\mathbf{y}, w|\mathbf{x})} = g(w, v). \tag{1.242}$$

Based on Theorem 2 from [**?**] and also Theorem 3.2 in [**?**], because $f(w; \mathbf{y}, \mathbf{x}, \lambda)$ is a function of w and g is smooth, the integral (**??**) is approximated by:

$$\begin{aligned}
D_{\tilde{P}}^{P} &= \int_{\mathbb{R}^d} e^{-\lambda f(w;\mathbf{y},\mathbf{x},\lambda)}g(w, v) && (1.243)\\
&= c_\lambda e^{-\lambda f(w^*;\mathbf{y},\mathbf{x},\lambda)}\left(\frac{\sqrt{\det D_f(w^*; \mathbf{y}, \mathbf{x}, \lambda)}}{\sqrt{\det D_f(w^*; \mathbf{y}, \mathbf{x}, \lambda)}}g(w^*, v) + \epsilon(\lambda)\right) && (1.244)\\
&= c_\lambda e^{-h(w^*;\mathbf{y},\mathbf{x})}\left(\frac{g(w^*, v)}{\sqrt{\det D_h(w^*; \mathbf{y}, \mathbf{x})}} + \frac{\epsilon(\lambda)}{\lambda}\right), && (1.245)
\end{aligned}$$

where $\epsilon(\lambda) = O(1/\lambda)$. Substituting h into the relations leads to the desired result.

If we set $v = \|w^*\|$, then by Theorem 3.1.3, $g(w^*, v)$ will be zero, and therefore the KL divergence between $\tilde{P}$ and P will be almost zero.

However, because h is non-convex, computing w^* is difficult. The following lemma provides an upper bound for h. This shows that instead of directly computing w^*, we can minimize a simple convex function in which w is more regularized. Interestingly, Lemma 3.2.3 relates robust learning to more regularization of the learning function. This issue has been shown in the context of robust SVM learning in [**?**].

Lemma 1.3. *Assume* $\sigma_w^2 = \sigma^2(\frac{1}{\sigma_w^2} + \sum_{i=1}^n \sigma_i^2)$ *and*

$$h'(w; \mathbf{y}, \mathbf{x}) = \frac{1}{2}\sum_{i=1}^{n} \frac{(y_i - w^T x_i)^2}{\sigma^2} + \frac{1}{2}\frac{\sigma_w^{-2}}{\sigma^2}\|w\|^2. \tag{1.246}$$

Then we have:

$$h(w; \mathbf{y}, \mathbf{x}) \leq h'(w; \mathbf{y}, \mathbf{x}) + n \log \sigma. \tag{1.247}$$

Proof. Using the inequality $\alpha + \beta \leq \alpha e^{\beta/\alpha}, \alpha \neq 0$, we have:

$$\sigma^2 \leq \sigma^2 + \sigma_i^2\|w\|^2 \leq \sigma^2 e^{\sigma_i^2\|w\|^2/\sigma^2}. \tag{1.248}$$

Using this in h, we get $h(w; \mathbf{y}, \mathbf{x}) \leq h'$, where

$$h' = \frac{1}{2}\sum_{i=1}^{n} \frac{(y_i - w^T x_i)^2}{\sigma^2} + n \log \sigma + \frac{\sigma_w^{-2}}{2}\|w\|^2. \tag{1.249}$$

Therefore, h' can be used to minimize h. However, it is important to know the difference between their solutions so that we can systematically use the solution of h' instead of the solution of h.

Theorem 1.8. *Let* w^* *be a regularized solution of* $\arg\min_w h(w; \mathbf{y}, \mathbf{x})$*, and let*

$$\tilde{w} = \arg\min_w h'(w, \mathbf{y}, \mathbf{x}). \tag{1.250}$$

Then we will have:

$$\frac{\|w^* - \tilde{w}\|}{\|w^*\|} \leq \frac{\sigma_w^2 \sigma_w^2}{\|w^*\|}(D_x + N_x) + \sigma_w^2 n \left\{\sum_{i=1}^{n} \frac{\sigma_i^2}{\sigma^2}\right\}. \tag{1.251}$$

Also, $\lim_{\sigma_1,\ldots,\sigma_n \to 0} \|w^* - \tilde{w}\| = 0$*, where*

$$D_x = \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\|D_{ij}\||\sigma_j^2 - \sigma_i^2|}{\sigma_i(w^*)\sigma_j(w^*)} \leq \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\|D_{ij}\|}{\sigma^4}|\sigma_j^2 - \sigma_i^2|, \tag{1.252}$$

$$N_x = \frac{1}{\sigma_w^2\sigma^2}\sum_{i=1}^{n} \frac{\|x_i y_i\|\sigma_i^2}{\sigma_i(w^*)} \leq \frac{1}{\sigma_w^2\sigma^4}\sum_{i=1}^{n} \|x_i y_i\|\sigma_i^2, \tag{1.253}$$

and $D_{ij} = \sigma^{-2}(x_j x_j^T + \sigma_j^2 I)(x_i y_i)$*. Note that* $\lim_{\|w^*\| \to \infty} \frac{D_x}{\|w^*\|^2} = \lim_{\|w^*\| \to \infty} N_x = 0$.

Proof. Let

$$h_\rho(w, \rho; \mathbf{y}, \mathbf{x}) = \frac{1}{2}\sum_{i=1}^{n}\left\{\frac{(y_i - w^T x_i)^2}{\rho_i} + \log(\rho_i)\right\} + \frac{1}{2\sigma_w^2}\|w\|^2. \tag{1.254}$$

The following problem is equivalent to the problem $\min_w h(w; \mathbf{y}, \mathbf{x})$:

$$\min_{w,\rho} h_\rho(w, \rho; \mathbf{y}, \mathbf{x}) \quad \text{s.t. } \rho_i = \sigma^2 + \sigma_i^2 \|w\|^2, \quad \forall i \in [1, n]. \tag{1.255}$$

The Lagrangian for problem (3-171) is:

$$\mathcal{L}(w, \rho; \theta) = h_\rho(w, \rho; \mathbf{y}, \mathbf{x}) - \theta^T(\rho - \sigma^2 - \sigma_I^2 \|w\|^2). \tag{1.256}$$

The first-order necessary condition for this problem is:

$$\nabla h_\rho(w^*, \rho^*; \mathbf{y}, \mathbf{x}) - \sum_{i=1}^n \theta_i \nabla(\rho_i^* - \sigma^2 - \sigma_i^2 \|w^*\|^2) = 0, \tag{1.257}$$

which can be written with respect to variables w, ρ_i as:

$$\sum_{i=1}^n \frac{x_i(x_i^T w^* - y_i)}{\rho_i^*} + \frac{w^*}{\sigma_w^2} + \sum_{i=1}^n \theta_i 2\sigma_i^2 w^* = 0, \tag{1.258}$$

$$\frac{1}{2} \frac{-(y_i - x_i^T w^*)^2}{(\rho_i^*)^2} + \frac{1}{2} \frac{1}{\rho_i^*} - \theta_i = 0. \tag{1.259}$$

Using these relations, we have:

$$V := \sum_{i=1}^n \frac{\sigma_i^2 (y_i - x_i^T w^*)^2}{(\rho_i^*)^2} = \sum_{i=1}^n \frac{\sigma_i^2}{\rho_i^*} - \sum_{i=1}^n 2\theta_i \sigma_i^2. \tag{1.260}$$

Using the second-order sufficiency conditions $\nabla^2 \mathcal{L} \succ 0$, we must have at least:

$$\nabla^2_{w,w} \mathcal{L} = \sum_{i=1}^n \frac{x_i x_i^T}{\rho_i^*} + \left(\frac{1}{\sigma_w^2} + \sum_{i=1}^n \theta_i 2\sigma_i^2 \right) I \succ 0, \tag{1.261}$$

$$\nabla^2_{\rho_i,\rho_i} \mathcal{L} = \frac{2(y_i - x_i^T w^*)^2 - (\rho_i^*)^2}{(\rho_i^*)^3} \succ 0, \quad \forall i. \tag{1.262}$$

Because we assume w^* is sufficiently regularized, $\sum_{i=1}^n \theta_i 2\sigma_i^2 \geq 0$. Therefore, the first condition in (3-174) is satisfied by (3-173):

$$V = \sum_{i=1}^n \frac{\sigma_i^2 (y_i - x_i^T w^*)^2}{(\rho_i^*)^2} \leq \sum_{i=1}^n \frac{\sigma_i^2}{\rho_i^*} \leq \sum_{i=1}^n \frac{\sigma_i^2}{\sigma^2}. \tag{1.263}$$

From the second condition in (3-174), we have:

$$(y_i - x_i^T w^*)^2 \geq \frac{1}{2}(\rho_i^*). \tag{1.264}$$

Using this, we have:

$$V = \sum_{i=1}^n \frac{\sigma_i^2 (y_i - x_i^T w^*)^2}{(\rho_i^*)^2} \geq \frac{1}{2} \sum_{i=1}^n \frac{\sigma_i^2}{\rho_i^*}. \tag{1.265}$$

Let

$$A = \sum_{i=1}^n \frac{x_i x_i^T}{\rho_i^*} + \left(\frac{1}{\sigma_w^2} + \sum_{i=1}^n \frac{\sigma_i^2}{\rho_i^*} \right) I. \tag{1.266}$$

Using (3-172), we have:

$$w^* = A^{-1} \sum_{i=1}^{n} \frac{x_i y_i}{\rho_i^*} + A^{-1} V w^*. \tag{1.267}$$

Let

$$B = \sum_{i=1}^{n} \frac{x_i x_i^T}{\sigma^2} + \sigma_w^{-2} I. \tag{1.268}$$

Minimizing the function h' in (3-168), we will have:

$$\tilde{w} = B^{-1} \sum_{i=1}^{n} \frac{x_i y_i}{\sigma^2}. \tag{1.269}$$

Using relations (3-178) and (3-179) and the triangle inequality, we will have:

$$\|w^* - \tilde{w}\| \leq \left\| A^{-1} \sum_{i=1}^{n} \frac{x_i y_i}{\rho_i^*} - B^{-1} \sum_{i=1}^{n} \frac{x_i y_i}{\sigma^2} \right\| + \|A^{-1} V w^*\|. \tag{1.270}$$

Let $W = B \sum_{i=1}^{n} \frac{x_i y_i}{\rho_i^*} - A \sum_{i=1}^{n} \frac{x_i y_i}{\sigma^2}$. Using (3-175) and the Cauchy-Schwarz inequality:

$$\left\| \sum_{i=1}^{n} \frac{x_i y_i}{\rho_i^*} - A \sum_{i=1}^{n} \frac{x_i y_i}{\sigma^2} \right\| \leq \|A^{-1}\| \|B^{-1}\| \|W\|. \tag{1.271}$$

Then:

$$\|w^* - \tilde{w}\| \leq \|A^{-1}\| \|B^{-1}\| \|W\| + \|A^{-1}\| n \left\{ \sum_{i=1}^{n} \frac{\sigma_i^2}{\sigma^2} \right\} \|w^*\|. \tag{1.272}$$

With some simplification, it is easy to see that

$$W = \sum_{i=1}^{n} \sum_{j=1}^{n} D_{ij} \left\{ \frac{1}{\rho_i^*} - \frac{1}{\rho_j^*} \right\} + \frac{1}{\sigma_w^2} \sum_{i=1}^{n} x_i y_i \left\{ \frac{\sigma^2 - \rho_i^*}{\sigma^2 (\rho_i^*)} \right\} \tag{1.273}$$

$$= \sum_{i=1}^{n} \sum_{j=1}^{n} D_{ij} \{\rho_j^* - \rho_i^*\} + \frac{1}{\sigma_w^2} \sum_{i=1}^{n} x_i y_i \{\sigma^2 - \rho_i^*\}, \tag{1.274}$$

where $D_{ij} = \frac{(x_j x_j^T + \sigma_j^2 I) x_i y_i}{\sigma^2}$. Therefore,

$$\|W\| \leq n \left(D_x + N_x \right) \|w^*\|^2. \tag{1.275}$$

It is easy to see that

$$\|A^{-1}\| = \lambda_{\max}(A^{-1}) \leq \sigma_w^2, \quad \|B^{-1}\| = \lambda_{\max}(B^{-1}) \leq \sigma_w^2. \tag{1.276}$$

Using these values, we conclude that

$$\frac{\|w^* - \tilde{w}\|}{\|w^*\|} \leq \sigma_w^2 \sigma_w^2 (D_x + N_x) \|w^*\| + \sigma_w^2 n \left\{ \sum_{i=1}^{n} \frac{\sigma_i^2}{\sigma^2} \right\}. \tag{1.277}$$

Computing the limit shows that $\lim_{\sigma_1, \ldots, \sigma_n \to 0} \|w^* - \tilde{w}\| = 0$.

Because $\tilde{w}$ is sufficiently close to w^*, we can use its value to approximate w^* for computing $\tilde{P}$, and based on the above theorem and Theorem 3.1.3, the distance between P and $\tilde{P}$ is sufficiently small for any value of $\mathbf{y}_U$. Since $e^{-h(w^*; \mathbf{y}, \mathbf{x})}$ and $e^{-h'(w^*; \mathbf{y}, \mathbf{x})}$ change much more than g with respect to $\mathbf{y}_U$, we use $\min_{\mathbf{y}_U} \min_w h'(w; \mathbf{y}, \mathbf{x})$ for the approximation.

Let K be the kernel matrix and also $L = (K + \sigma_w^{-2} I)^{-1} K (K + \sigma_w^{-2} I)^{-1}$. Then the following theorem determines the estimate of the objective function:

Theorem 1.9. *Let*

$$\tau = \mathbf{y}_L^T(L_{ll} - L_{lu}L_{uu}^{-1}L_{ul})\mathbf{y}_L, \tag{1.278}$$

and $\kappa_i = (\sigma^2 + \sigma_i^2\tau)^{-1}$, $\Upsilon = \mathrm{diag}(\kappa)$, $S = \Upsilon - \Upsilon n\sigma_w^2(K^{-1} + \sigma_w^2\Upsilon)^{-1} \circ \Upsilon$. *We have:*

$$\tilde{P}(\mathbf{y}|\mathbf{x}) \propto \exp\left(-\frac{1}{2}\mathbf{y}^T S\mathbf{y}\right). \tag{1.279}$$

Proof. To compute $\tilde{P}$, similar to (3-163), we first compute the estimate $v^2 = \|\tilde{w}\|^2$, where $\tilde{w} = \arg\min_w h'(w; \mathbf{y}, \mathbf{x})$. Based on Lemma 3.2.3, $\tilde{w} = (XX^T + \sigma_w^{-2}I)^{-1}X\mathbf{y}$, where $X = [x_1, x_2, \dots, x_n]$. Therefore,

$$v^2 = \|\tilde{w}\|^2 = \mathbf{y}^T X^T(XX^T + \sigma_w^{-2}I)^{-1}(XX^T + \sigma_w^{-2}I)^{-1}X\mathbf{y}. \tag{1.280}$$

Using L and the equality [**?**]

$$PB^T(BPB^T + R)^{-1} = (P^{-1} + B^TR^{-1}B)^{-1}B^TR^{-1}, \tag{1.281}$$

we have:

$$\|\tilde{w}\|^2 = \mathbf{y}^T(X^TX + \sigma_w^{-2}I)^{-1}X^TX(X^TX + \sigma_w^{-2}I)^{-1}\mathbf{y} \tag{1.282}$$

$$= \mathbf{y}_L^T L_{ll}\mathbf{y}_L + 2\mathbf{y}_L^T L_{lu}\mathbf{y}_U + \mathbf{y}_U^T L_{uu}\mathbf{y}_U. \tag{1.283}$$

Minimizing this with respect to $\mathbf{y}_U$, we get:

$$\mathbf{y}_U = -H_{uu}^{-1}H_{ul}\mathbf{y}_L, \tag{1.284}$$

and similarly,

$$v^2 = \|\tilde{w}\|^2 \geq \mathbf{y}_L^T(L_{ll} - L_{lu}L_{uu}^{-1}L_{ul})\mathbf{y}_L \equiv \tau. \tag{1.285}$$

To compute $\tilde{P}(\mathbf{y}|\mathbf{x})$, we first compute:

$$\tilde{P}(\mathbf{y}, w|\mathbf{x}) = \tilde{P}(\mathbf{y}|\mathbf{x}, w)P(w) \tag{1.286}$$

$$= P(w)\prod_{i=1}^{n}\mathcal{N}(y_i; w^Tx_i, \sigma^2 + \sigma_i^2\tau^2) \tag{1.287}$$

$$\propto \exp\left(-\frac{1}{2}\left\{\sum_i \kappa_i(y_i - w^Tx_i)^2 + \sigma_w^{-2}\|w\|^2\right\}\right). \tag{1.288}$$

Let

$$M = n\sigma_w^{-2}I + \sum_i \kappa_i x_i x_i^T, \quad \nu = M^{-1}\sum_i \kappa_i y_i x_i. \tag{1.289}$$

By completing the square, we have:

$$\tilde{P}(\mathbf{y}|\mathbf{x}) = \int \tilde{P}(\mathbf{y}, w|\mathbf{x})dw \propto \exp\left(-\frac{1}{2}\left\{\sum_i \kappa_i y_i^2 - \|\nu\|^2 M\right\}\right). \tag{1.290}$$

With some simplification,

$$\sum_i \kappa_i y_i^2 - \|\nu\|^2 M = \mathbf{y}^T\left\{\Upsilon - \Upsilon X^T M^{-1} X\Upsilon\right\}\mathbf{y}. \tag{1.291}$$

Using the Woodbury identity and $K = X^TX$, we get:

$$X^TM^{-1}X = \sigma_w^2\left\{X^TX - X^TX(\sigma_w^{-2}\Upsilon^{-1} + X^TX)^{-1}X^TX\right\}. \tag{1.292}$$

Again using Woodbury, we get:

$$S = \Upsilon n\sigma_w^2(K^{-1} + \sigma_w^2\Upsilon)^{-1} \circ \Upsilon. \tag{1.293}$$

The final objective function for the Probabilistic Minimax Active Robust Learning (PMARL) approach is:

$$Q = \underset{\{Q,|Q|=b\}}{\arg\min} \max_{y_Q} \min_{y_U} \mathbf{y}^T S\mathbf{y}. \tag{1.294}$$

Fortunately, because this objective function is quadratic, we can easily compute Q when $b = 1$.

There are many real-world applications for the robust active learning problem. One of these applications is active learning on distributions. The objective function (**??**) can be used directly for active learning on distributions. For this purpose, it is sufficient to set $\sigma_i = \beta n_i^{-1/2}$ and $\Upsilon = \mathrm{diag}(1 \oslash (\sigma^2 + \sigma_I^2\tau))$, where β is a parameter. We call this method Probabilistic Minimax Active Robust Learning on Distributions using Similarity (PMARLDS).

1.2.4 Robust Active Learning on Distributions using Bayesian Learning of Kernel Embeddings

To clarify the difference, in this section we call the points on which learning is performed "examples" and the points that are members of a sample drawn from a distribution "points." We have a number of distributions $P_i, i \in \{1, ..n\}$ as examples. For each distribution P_i, we have a sample $x_i = \{x_{i1}, x_{i2}, \ldots, x_{in_i}\}$ drawn from it. Each member of this sample is a vector $x_{iu} \in \mathbb{R}^d, \forall u \in [1, n_i]$.

The kernel embedding of distribution P_i using the kernel function k is:

$$\mu^*_{P_i} := \int\int k(x, .)dP_i(z), \tag{1.295}$$

where k is a continuous, bounded, and positive definite kernel function [**?**]. The empirical kernel embedding for distribution P_i is:

$$\hat{\mu}_{P_i} = \frac{1}{n_i}\sum_{j=1}^{n_i} k(x_{ij}, .). \tag{1.296}$$

Let $\mathcal{H}$ be the Hilbert space associated with the kernel function k. $\mu(x)$ means the evaluation of μ at x. Specifically, for a given value x, the value

$$\hat{\mu}_{P_i}(x) = \frac{1}{n_i}\sum_{j=1..n_i} k(x_{ij}, x) \tag{1.297}$$

is the evaluation of the empirical embedding of distribution P_i at x.

We need to compute the value $p(\mathbf{y}|\hat{\boldsymbol{\mu}}_P)$. The embedding $\hat{\boldsymbol{\mu}}_P$ is not the best estimate for the distribution P [**?**]. Recently, it was shown that we can obtain more accurate kernel embeddings with an uncertainty measure in the Hilbert space using a Bayesian framework [**?**]. We use this framework to consider the uncertainty in the kernel embedding estimate for the distribution. Assume k_θ is a positive definite kernel function with parameter θ. The Bayesian kernel embedding proposes to define the prior probability on μ^* to ensure that the kernel embedding lies in the corresponding kernel space, i.e., $\mu^* \in \mathcal{H}_k$, as follows [?]:

$$\mu^*|\theta \sim \mathcal{GP}(0, r_\theta(.,.)), \quad r_\theta(x,y) = \int k_\theta(x,u)k_\theta(u,y)du, \tag{1.298}$$

where $\mathcal{GP}$ is a Gaussian process. The choice of r_θ guarantees that $\mu^*_{P_i} \in \mathcal{H}$ [?]. θ is the kernel function parameter, and henceforth we will omit it for simplicity.

The following lemma determines the probability of a label for example P_i given the kernel embedding value for example P_i at each location $x_{ti} \in x_i, t \in [1, n_i]$.

Lemma 1.4. *Assume r is the kernel function according to (3-184), and $r_{xx} = r(x,x)$, and $R_i = [r(x_{ti}, x_{vi})]_{t,v=1}^{n_i}$. Also,*

$$R_i^x = [r(x, x_{1i}), r(x, x_{2i}), \ldots, r(x, x_{n_i})]^T. \tag{1.299}$$

Furthermore, assume

$$B_i = (R_i + (\rho^2/n_i)I_n)^{-1}R_i^x, \quad C_i = r_{xx} - R_i^{xT}(R_i + (\rho^2/n_i)I_n)^{-1}R_i^x. \tag{1.300}$$

Assuming a linear model $y_i = w^T\mu^*_{P_i} + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$, we have:

$$P(y_i|\hat{\mu}_{P_i}(\mathbf{x}_i), w) \propto \sigma_i(w)^{1/2} \exp\left(-\frac{1}{2\sigma_i(w)}(y_i - w^T B_i \hat{\mu}_{P_i}(\mathbf{x}_i))^2\right), \tag{1.301}$$

where $\sigma_i(w) = \sigma^2 + w^T C_i w$.

Proof. Based on Theorem 1.1.2, the variance of the function $P(\hat{\mu}_{P_i}(x)|\mu^*_{P_i}(x))$ is proportional to $n_i^{-1/2}$. Therefore, we can assume that this function is:

$$P(\hat{\mu}_{P_i}(x)|\mu^*_{P_i}(x)) = \mathcal{N}(\hat{\mu}_{P_i}(x); \mu^*_{P_i}(x), \rho^2/n_i), \tag{1.302}$$

where ρ is a parameter. This has also been proven in [?] based on the central limit theorem.

Using a simple result in Gaussian processes [?], the posterior distribution is:

$$P(\mu^*_{P_i}(x)|\hat{\mu}_{P_i}(\mathbf{x}_i)) = \mathcal{N}(\mu^*_{P_i}(x); B_i^T\hat{\mu}_{P_i}(\mathbf{x}_i), C_i). \tag{1.303}$$

The quantity $\mu^*_{P_i}$ is unknown. Therefore, by marginalizing over $\mu^*_{P_i}$ along with the linear model assumption $y_i = w^T\mu^*_{P_i} + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$, we have:

$$\begin{aligned}
P(y_i|\hat{\mu}_{P_i}(\mathbf{x}_i), w) &= \int P(y_i|\mu^*_{P_i}, w)P(\mu^*_{P_i}|\hat{\mu}_{P_i}(\mathbf{x}_i))d\mu^*_{P_i} && (1.304)\\
&= \int \mathcal{N}(y_i; w^T\mu^*_{P_i}, \sigma^2)\mathcal{N}(\mu^*_{P_i}; \hat{\mu}_{P_i}(\mathbf{x}_i), C_i)d\mu^*_{P_i} && (1.305)\\
&\propto (\sigma_i(w))^{-1/2}\exp\left(-\frac{1}{2}\frac{(y_i - w^T B_i\hat{\mu}_{P_i}(\mathbf{x}_i))^2}{\sigma_i(w)}\right). && (1.306)
\end{aligned}$$

If the distribution P_i has a large covariance C_i, meaning that $w^T C_i w$ is large, its effect on the classifier will be small. Therefore, the probability of selecting P_i for querying in active learning should be low. Looking at relation (3-185), we can gain insight into when this happens. This occurs when the normalized value of R_i^x, i.e., $(R_i + (\rho/n_i)I_n)^{-1/2}R_i^x$, is approximately orthogonal to w in the Hilbert space.

To compute $P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P)$, we have:

$$P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x})) = \int P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}), w)p(w)dw. \tag{1.307}$$

Similar to 1.2.3, this integral is intractable, and we need to find a way to efficiently compute its estimate.

The problem now is how to find a tight lower bound for $w^T C_i w + \sigma^2$ for τ_i. Here, we estimate the minimum value for $w^T C_i w$ from simpler conditions. In these simpler conditions, we assume there is no uncertainty for any distribution, and therefore the distribution is determined by its mean in the Hilbert space. Naturally, when C_i is constant, considering the uncertainty for each distribution P_i will not increase the value of $w^T C_i w$, because this itself is a measure of the complexity of w. The following theorem defines the lower bound τ_i for $w^T C_i w$.

Theorem 1.10. *Assume U is unlabeled data and L is labeled data for which we have the known labels $\mathbf{y}_L$. Without loss of generality, assume $\mathbf{y} = (\mathbf{y}_L, \mathbf{y}_U)$, where $\mathbf{y}_U$ is unknown. Furthermore, assume that kernel embedding through samples simplifies $P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}))$. Then $P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}))$ in relation (3-187) has the following tight lower bound:*

$$\tilde{P}(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x})) \leq P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x})). \tag{1.308}$$

This lower bound is proportional to:

$$\tilde{P}(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x})) \propto \exp\left(-\frac{1}{2}\mathbf{y}^T S \mathbf{y}\right), \tag{1.309}$$

where

$$S = \Upsilon - \sigma_w^2 \Upsilon \left\{ K_{\mu B} - K_{\mu B}\left(\sigma_w^{-2}\Upsilon^{-1} + K_{\mu B}\right)^{-1} K_{\mu B} \right\}. \tag{1.310}$$

Assume $\mu_{R_i} = (R_i + (\tau^2/n_i)I_n)^{-1}\hat{\mu}_{P_i}(x_i)$. Then $K_{\mu B}$ is a matrix with elements

$$K_{\mu B}(i,j) = [\mu_{R_i}^T R_{ij} \mu_{R_j}], \tag{1.311}$$

and

$$\Upsilon = \mathrm{diag}(1 \oslash \tau), \quad \tau_i = \mathbf{y}_L^T (H_{ll} - H_{lu} H_{uu}^{-1} H_{ul}) \mathbf{y}_L, \tag{1.312}$$

where

$$L_t = (K_{\mu B} + \sigma_w^{-2} I)^{-1} E_t (K_{\mu B} + \sigma_w^{-2} I)^{-1}, \quad E_t = K_{\mu B} - P_t^T (R_t + (\tau^2/n_t)I_n)^{-1} P_t, \tag{1.313}$$

and $P_t = [R_{ti}\mu_{R_i}]_{i=1}^n \in \mathbb{R}^{n_t \times n}$. Then we have: $\tau_i = \mathbf{y}_l^T \{L_i^{ll} - L_i^{ul^T}(L_i^{uu})^{-1} L_i^{ul}\}\mathbf{y}_l$. This lower bound is tight in the sense that there exists a label $y_U \in [-1,1]^{n_u}$ for which the value of $P(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}))$ is equal to this lower bound.

Proof. To compute $\tilde{P}$, we need to evaluate the value $\tilde{v}_i^2 = \tilde{w}^T C_i \tilde{w}$. First, we need to estimate $\tilde{w}$. Assume $\hat{\mu}_B(\mathbf{x}) = [\hat{\mu}_{B_1}(x_1), \ldots, \hat{\mu}_{B_n}(x_n)]^T$, and $K_{\mu B} = \hat{\mu}_B(\mathbf{x})\hat{\mu}_B(\mathbf{x})^T$. Based on Lemma 3.2.3, we have:

$$\tilde{w} = \arg\min_w h'(w; \mathbf{y}, \hat{\mu}_P(\mathbf{x})) = (K_{\mu B} + \sigma_w^{-2} I)^{-1} \hat{\mu}_B(\mathbf{x})\mathbf{y}. \tag{1.314}$$

By substituting $\hat{\mu}_{B_i}(x_i)$ using μ_{R_i} and $R_{i*}^T R_{j*} = R_{ij}$, we have $K_{\mu B} = [\mu_{R_i}^T R_{ij} \mu_{R_j}]$. Let $K_{\text{reg}} = K_{\mu B} + \sigma_w^{-2} I$. We have:

$$\|\tilde{w}\|^2 C_t = \{\hat{\mu}_B(\mathbf{x})\mathbf{y}\}^T (K_{\text{reg}})^{-1} C_t (K_{\text{reg}})^{-1} \hat{\mu}_B(\mathbf{x})\mathbf{y}. \tag{1.315}$$

Let $E_t = \hat{\mu}_B(\mathbf{x})^T C_t \hat{\mu}_B(\mathbf{x})$. Using equality (3-180) and L_t, it follows that:

$$\begin{aligned} \|\tilde{w}\|^2 C_t &= \mathbf{y}^T (K_{\mu B} + \sigma_w^{-2} I)^{-1} E_t (K_{\mu B} + \sigma_w^{-2} I)^{-1} \mathbf{y} && (1.316) \\ &= \mathbf{y}^T L_t \mathbf{y} = \mathbf{y}_L^T L_t^{ll} \mathbf{y}_L + 2\mathbf{y}_L^T L_t^{lu} \mathbf{y}_U + \mathbf{y}_U^T L_t^{uu} \mathbf{y}_U. && (1.317) \end{aligned}$$

By minimizing the last equality with respect to the unknown values $\mathbf{y}_U$, we get:

$$\|\tilde{w}\|^2 C_t \geq \mathbf{y}_L^T L_t^{ll} \mathbf{y}_L - \mathbf{y}_L^T L_t^{lu} (L_t^{uu})^{-1} L_t^{ul} \mathbf{y}_L \equiv \tau_t. \tag{1.318}$$

Substituting B_i, $\hat{\mu}_B(\mathbf{x})$, and using μ_{R_i}, we obtain:

$$\begin{aligned} E_t(i,j) &= \hat{\mu}_B(\mathbf{x})^T C_t \hat{\mu}_B(\mathbf{x})(i,j) && (1.319) \\ &= \hat{\mu}_{P_i}(x_i)^T B_i C_t B_j^T \hat{\mu}_{P_j}(x_j) && (1.320) \\ &= \hat{\mu}_{P_i}(x_i)^T \left\{ (R_i + (\rho^2/n_i) I_n)^{-1} R_{i*} \right\} C_t && (1.321) \\ &\quad \left\{ (R_j + (\rho^2/n_i) I_n)^{-1} R_{j*} \right\}^T \hat{\mu}_{P_j}(x_j) && (1.322) \\ &= \mu_{R_i}^T R_{i*} C_t R_{j*}^T \mu_{R_j}. && (1.323) \end{aligned}$$

The term $R_{i*} C_t R_{j*}^T$ can be simplified as:

$$\begin{aligned} R_{i*} C_t R_{j*}^T &= R_{i*} \left\{ r_{**} - R_{t*}^T (R_t + (\rho^2/n_t) I_n)^{-1} R_{t*} \right\} R_{j*}^T && (1.324) \\ &= R_{ij} - R_{it} (R_t + (\rho^2/n_t) I_n)^{-1} R_{tj}. && (1.325) \end{aligned}$$

Then:

$$E_t(i,j) = \mu_{R_i}^T \left\{ R_{ij} - R_{it} (R_t + (\rho^2/n_t) I_n)^{-1} R_{tj} \right\} \mu_{R_j} = K_{\mu B}(i,j) - \mu_{R_i}^T R_{it} (R_t + (\rho^2/n_t) I_n)^{-1} R_{tj} \mu_{R_j}. \tag{1.326}$$

Using P_t, we have:

$$E_t = K_{\mu B} - P_t^T (R_t + (\rho^2/n_t) I_n)^{-1} P_t. \tag{1.327}$$

Therefore,

$$\tilde{P}(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}), w) = \prod_{i=1}^{n} \tilde{P}(y_i|\hat{\mu}_{P_i}(\mathbf{x}_i), w) \propto \exp\left(-\frac{1}{2} H\right), \tag{1.328}$$

where

$$H = \sum_{i=1}^{n_i} \kappa_i (y_i - w^T \hat{\mu}_{B_i}(\mathbf{x}_i))^2. \tag{1.329}$$

Similar to Theorem 3.3.4, by setting

$$M = \sigma_w^{-2} I + \sum_{i=1}^{n_i} \kappa_i \hat{\mu}_{B_i}(\mathbf{x}_i) \hat{\mu}_{B_i}(\mathbf{x}_i)^T, \tag{1.330}$$

completing the square and simplifying using $K_{\mu B} = \hat{\mu}_B(\mathbf{x}) \hat{\mu}_B(\mathbf{x})^T$, it follows that:

$$\begin{aligned} \tilde{P}(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x})) &= \int \tilde{P}(\mathbf{y}|\hat{\boldsymbol{\mu}}_P(\mathbf{x}), w) P(w) dw && (1.331) \\ &\propto \exp\left\{ -\frac{1}{2} \mathbf{y}^T S \mathbf{y} \right\}. && (1.332) \end{aligned}$$

This result is very intuitive because it essentially expresses the similarity between two distributions (i,j) for selecting the label of distribution t based on a common criterion, i.e., the metric present in distribution t.

Chapter 2

Experiments

In this chapter, we evaluate the performance of the proposed methods. First, we evaluate the performance of the proposed functional gradient approach to Probabilistic Minimax Active Learning. Then, in Section 3.0.4, we evaluate the performance of the proposed active learning on distributions method.

2.1 Experiments Related to the Functional Gradient Approach

The goal of these experiments is to evaluate the efficiency of the proposed method compared to other state-of-the-art active learning strategies. Therefore, a set of experiments was designed to determine whether the proposed method based on Probabilistic Minimax Active Learning provides a better selection mechanism than other active learning methods, as well as boosting-based methods. For this purpose, we conducted experiments on a large set of widely used datasets in machine learning papers and problems.

2.1.1 Experiments on Various Datasets

In this section, a set of experiments is conducted to demonstrate the efficiency of the proposed method compared to other active learning strategies on a large number of datasets. The goal is to show the effectiveness of the proposed method in selecting examples for querying based on the functional approach to active learning.

In these experiments, the following methods were examined:

- **RANDOM**: Random sampling, which selects examples randomly.
- **UNCERTAIN**: Uncertainty sampling, which selects examples based on the degree of uncertainty about their label.
- **MARGIN**: This method selects the example closest to the current classifier boundary, regardless of its label [**?**]. Here, the SVM classifier is used, implemented using the LIBSVM library.
- **HARMONIC**: The Gaussian harmonic method is a semi-supervised and active learning method [**?**].
- **SPCTL2BOOST**: The proposed method.

To evaluate the algorithm, we selected a diverse set of datasets from various applications. These datasets include UCI datasets such as Breast Cancer, Bupa Liver, CODRNA, COVTYPE, ECOLI, Four Class, German Numer, Gisette, Glass, IJCNN, Image Segment, Ionosophere, Mushrooms, Pima Diabettes, and Sonar. We also used other datasets provided with the libsvm tools [?], including german.numer, IJCNN1, phishing, liver-disorders, a1a, a2a, a3a, a4a, and a5a. The Gisette dataset is another dataset used in the NIPS feature challenge [?]. Some other datasets were obtained from other well-known sources [?, ?]. Table 4.1 provides details of the datasets used.

In each of these experiments, the dataset was randomly divided into two separate subsets: one for training and one for testing. The test subset was used throughout the experiments to evaluate classifier accuracy. Active learning starts with one randomly selected point from each class until the labeling budget is exhausted. When a larger budget did not affect accuracy, we reduced the labeling budget as needed. At each iteration of active learning, an example is selected and its label is used to retrain the classifier and evaluate the classifier on the test subset. The LIBSVM library was used for training the classifier and comparing results for all methods [?]. The parameter C in LIBSVM was set to 1. The bandwidth of the RBF kernel was determined using the median heuristic [?]. The results of the experiments are presented in the figures.

The performance of the RANDOM method is sometimes acceptable in the early stages of some datasets and can provide good accuracy at the end. Some examples have labels that are correctly identified by the current classifier and do not change the current classifier boundary; these are called useless examples [?]. When all examples are useless, active learning approaches select these examples, wasting the labeling budget. The MARGIN approach, which is the most popular active learning approach for SVM classifiers, performs very well on some datasets. However, it also performs very poorly on some datasets, such as Breast and IJCNN. The performance of the Harmonic method is acceptable on some datasets but significantly decreases on others. Finally, as seen in the experimental results, the simple form of the proposed method, SPCTL2BOOST, generally provides better performance than other methods.

Table 4.2 shows the time spent for each method. For this purpose, we needed to calculate the average time spent in the querying method for each algorithm. As seen in Table 4.2, the time consumption of the proposed algorithm is often comparable to that of MARGIN. The time spent by the proposed method is often less than that spent by UNCERT. The time spent by the HARMON method is greater than that of UNCERT and is therefore not shown in the table.

2.1.2 Comparison with Boosting-Based Approaches

In this section, we compare the proposed method with other boosting-based methods. Therefore, a set of experiments was designed with a focus on this comparison.

As previously mentioned, the proposed method can be seen as a boosting algorithm in which a set of weak classifiers learns from the data. This subsection compares the proposed method with the boosting-based active learning method. As presented in [?], the boosting approach to active learning selects points that have the minimum distance to the current ensemble of classifiers in the boosting classification.

To determine whether the functional gradient approach to Probabilistic Minimax Active Learning is more effective than other boosting-based methods, we compared three algorithms using a common weak classifier with the same number of functional components/classifiers. The method used L2Boosting from [?] to obtain the FUNCGRDSTAL active learning method. For all methods, we used the datasets from the well-known work [?]. Here, the boosting-based method is named MARGINBOOST. The settings for these experiments are exactly the same as the previous experiments. Figure 4.4 shows the results of the experiments. As is clear, the proposed method SPCTL2BOOST is more efficient than the other two methods. The results show that the proposed method is more efficient because it selects more informative and representative points.

2.1.3 Comparison with Distribution Discrepancy Methods

The goal of this section is to compare the proposed method with active learning algorithms that measure the difference between labeled, unlabeled, and query sets. Specifically, the proposed algorithm is compared with the following two new methods:

- **BMDR**: The approach proposed in [?].
- **EXPREPINF**: The approach proposed in [?].

For these algorithms, parameters were set according to their original papers. All settings, as well as the parameter settings for the proposed method, were determined as in the previous experiments. We repeated the experiments ten times and reported the average results. Each iteration of active learning took much longer than the experiments in the previous step. The reason for this was that both BMDR and EXPREPINF methods required solving a constrained quadratic problem in each iteration. The constrained quadratic problems were solved using Mosek. Figure 4.5 shows the experimental results compared to the distribution discrepancy methods. As seen in Figure 4.5, the results of the proposed method show improvement compared to these methods.

2.1.4 Distributional Datasets

In this section, we examine the results of active learning on distribution experiments. First, to demonstrate the efficiency of the proposed method in controlled conditions, we conducted experiments on a synthetic dataset. Then, we conducted experiments on real-world data. In each experiment, we compared the results of the proposed method with the following methods:

- **DISTRAND**: This method randomly selects distributional examples.
- **DISTQUIRE**: We compute the mean of each distribution in the input space. Then, for each query, points are selected using the Quire method [?].
- **DISTHARM**: The mean of each distribution is computed, and then examples are selected using the Gaussian harmonic method [?]. This method performs active learning along with semi-supervised learning through harmonic functions.
- **DISTMARGIN**: Using the computed means for each distribution, margin-based active learning in the SVM classifier selects the most informative points [?].
- **HILBQUIRE**: Here, we proposed the HILBQUIRE method. In this proposed method, points are selected based on the QUIRE method [?]. In this method, the kernel matrix between examples in QUIRE is computed using the mean embedding of distributions [?].

In all experiments, each dataset was randomly divided into two separate datasets for training and testing. At each stage of active learning, we trained the classifier on the labeled data and reported the accuracy on the test data. We implemented the SMM classifier [?] using the LIBSVM package [?] for training and testing on distributional data. In these experiments, we used the empirical expectation kernel along with the RBF kernel between distributions using relation (2-1).

For the synthetic distributional dataset, we used the dataset presented in [?]. We call this dataset GAUSSDIST. Two classes of Gaussian distributions are considered for both positive and negative classes. The mean parameters of these Gaussian distributions are normally distributed between (1, 1, ..., 1) and (2, 2, ..., 2) with covariance $\Sigma = 0.5I$. The covariance of these distributions was obtained by a Wishart distribution with covariance $\Sigma_+ = 0.6I$ and $\Sigma_- = 1.2I$ with 10 degrees of freedom.

We randomly selected 30% of the samples for testing. The training set consisted of 150 distributions for the positive and negative classes. We set the kernel parameters for the input level and the second level to $\gamma = \gamma_{is} = 0.1$, and the regularization parameter to $\lambda = 2^{-7}$. Each experiment was repeated 50 times.

To demonstrate the efficiency of the proposed active learning method, which considers uncertainty in the kernel embedding estimate of distributions, we designed an experiment with significantly different sample sizes for each distribution. For this purpose, a sample size of 5 or 100 was randomly selected for each distribution. The experimental results are shown in Figure 4.6. When the number of sampled points from each distribution is different, our proposed method PMARLDB performs much better than other methods. The reason for this could be that some distributions with large sample sizes bias the learning toward themselves.

By directly modeling the uncertainty in estimating distributions, the accuracy of the proposed method increases in conditions of high uncertainty. Our proposed approach uses uncertainty information in both small and large sample size conditions to select better distributions. Even when the number of sampled points from each distribution is equal and small, our proposed method outperforms other methods. Although HILBQUIRE performs well in conditions of different sizes, its accuracy is not good in small sample size conditions. This situation usually occurs when there is high uncertainty in the kernel embedding estimate for the distributions used in HILBQUIRE.

2.1.5 Real-World Datasets

Handwritten Digit Recognition Here, we present the experimental results on the USPSDIST handwritten digit dataset [?]. This dataset is a distributional dataset created from images of digits zero to nine. The points are obtained using basic transformations (translation, resizing, rotation) on the images. Each image of a digit is a 16×16 matrix. The prior probability on the transformation parameters of equivalent classes determines the distributions.

We used two sets of parameters for resizing and translation, one for the horizontal axis and one for the vertical axis. For rotation, we used a single parameter for the rotation angle. Similar to [?], the values $\mathcal{N}([1,1], 0.1I_2)$, $\mathcal{N}([0,0], 5I_2)$, and $\mathcal{N}(0,\pi)$ were used for scaling, translation, and rotation, respectively. We randomly selected a subset of class pairs 3-8, 6-9, 3-4, and 1-8. In each of the mentioned classification tasks, 100 distributions were used. For each distribution P_i, the number of random points n_i was generated according to the same method as in [?]. The value n_i was set to $\max(1, \mathrm{round}(\mathcal{N}(2, 10)))$.

Active learning was started with two labeled examples and terminated with 50 examples. The experiments were repeated 50 times for each dataset. The results are shown in Figure 4.9.

As observed in this figure, even with low variance in the sample sizes for each distributional point, our proposed method implemented in PMARLDB gives better results than other methods. Comparing the PMARLDB method with random sampling shows that PMARLDB selects distributional points whose labels significantly improve classifier performance.

Bag of Words In this section, we present the experiments on several text datasets. To represent each document, we used the Bag of Words representation. We assume that each document can be represented by a distribution from which words are sampled. Each word can be represented by a vector, and each document by a distribution of these vectors. Because each document contains a variable number of words, we have sets of different sizes for the documents.

Here, we used word2vec [**?**] to convert words to vectors. In the experiments, we used the 20 Newsgroups dataset, which contains 20 different classes. Because this dataset is a multi-class dataset, we selected a number of class pairs that are more difficult to classify. We also selected fewer simpler tasks to demonstrate the performance of the proposed method on both hard and easy tasks.

Because some documents in this dataset contain a very large number of words, we removed documents with more than 100 words. Then, we randomly selected 300 documents for each class. Figure ? shows the experimental results. As seen in this figure, the proposed method PMARLDB provides better results compared to almost all methods and is logically better than random sampling.

2.1.6 Analysis of the Effect of Bayesian Learning of Kernel Embeddings of Distributions

A natural question that arises is whether using Bayesian learning of kernel embeddings of distributions in the PMARLDB active learning on distributions approach improves its results, or whether merely considering the inaccuracy of examples in robust active learning in the distribution space yields the improvements.

On the other hand, the proposed active learning method in 1.2.3 can be used to attack the active learning on distributions problem. To characterize the uncertainty in examples, Theorem 1.1.2 states the uncertainty in the empirical mean embedding of distributions. This uncertainty can be expressed using a Gaussian distribution with mean equal to the empirical mean embedding of distributions and variance proportional to $1/\sqrt{n}$. This approach is essentially a weaker version of the PMARLDB method, in which Bayesian learning of kernel embeddings is not considered.

In other words, our question is whether the robust active learning method from 1.2.3 using Theorem 1.1.2 for considering uncertainty in examples is effective for selecting distributional points.

Theorem 1.1.2 essentially states that the estimate of distributions using the mean embedding of distributions in Hilbert space has a variance of order $O(n^{-1/2})$. Therefore, we can use the formulation from 1.2.3 for selecting examples. As previously mentioned, we call this method PMARLDS. In this section, we compare this method with the main method to demonstrate the effectiveness of the proposed method. To this end, we conducted experiments on the 20 Newsgroups dataset to compare PMARLDB and PMARLDS. The results are shown in Figure 4.10. In this figure, it is clear that the proposed main method PMARLDB is always better than PMARLDS. PMARLDB shows significant improvement due to the simultaneous Bayesian learning of kernel embeddings.

2.2 Conclusion

In this chapter, we presented a general and theoretical active learning approach. This approach is particularly useful when there is uncertainty in the examples along with uncertainty about the unknown labels of the majority of examples. When examples are inaccurate, it was shown that considering this uncertainty during sample selection in active learning is very effective. Our main goal was to select distributional examples with different sizes from each distribution. To this end, we simultaneously performed Bayesian learning of distribution embeddings in kernel space during active learning, which led to an intuitively meaningful formulation for comparing and selecting distributions. Experiments on distributional and real-world datasets demonstrated the efficiency of the proposed method.

One of the limitations of kernel-based learning on distributions is that the samples drawn from the distributions must be stored. This gradually becomes a major obstacle when the number of distributions and sampled points increases. In the future, we plan to use linear feature methods to reduce the storage requirement for large-scale datasets.

Tables and Figures

Table 2.1: Dataset Characteristics

Dataset	# of Features	# of Examples
a1a	123	1605
a2a	123	2265
a3a	123	3185
a4a	123	4781
a5a	123	6414
Breast Cancer	10	683
Bupa Liver	7	345
CORDNA	8	59525
FOUR Class	2	862
GERNMAN Numer	24	1000
GISETTE	5000	6000
GLASS	10	214
IJCNN	22	49990
IMAGESEGMENT	19	2310
Ionoshpere	34	351
MUSHROOMS	22	8124
PIMA Diabettes	8	768
SONAR	60	208
phishing	68	11055
liver-disorders	5	145

Table 2.2: Time spent by active learning methods on different datasets

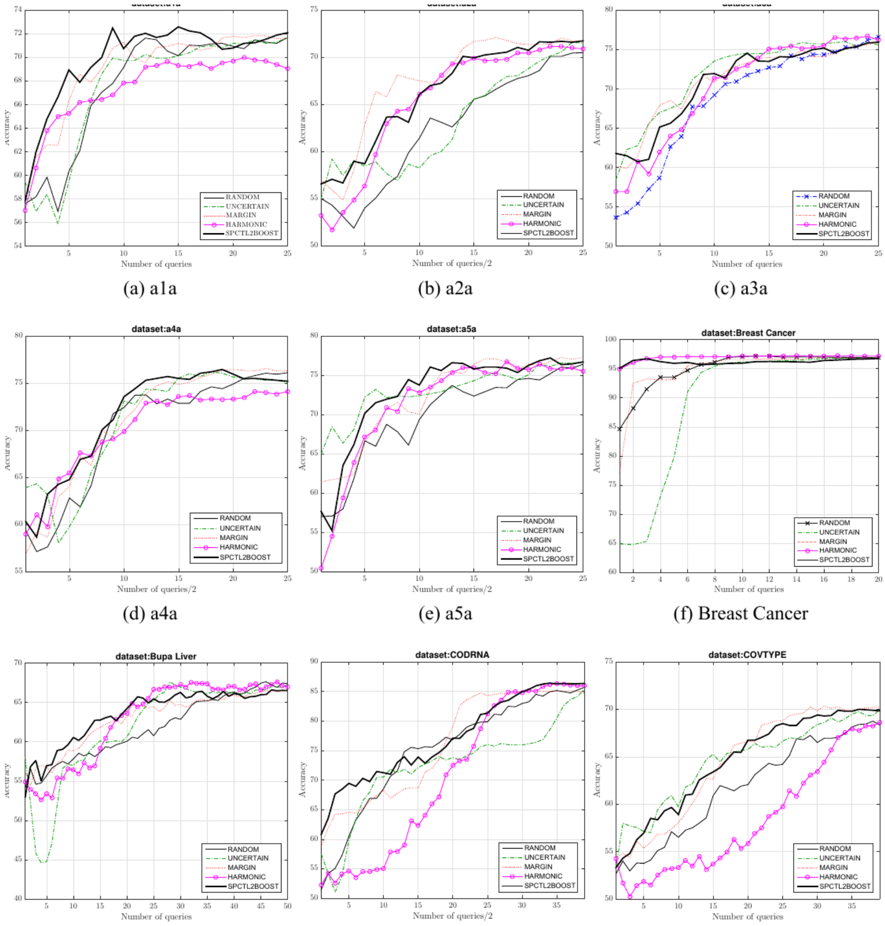
Dataset	RANDOM	SPCTL2BOOST	MARGIN	UNCERT
a1a	0.69	8.75	2.29	44.07
a2a	0.69	8.69	2.29	43.24
a3a	0.71	8.66	2.30	43.27
a4a	0.69	8.78	2.28	43.35
a5a	0.69	8.89	2.30	43.32
breast	0.77	24.25	1.98	132.92
Bupa	0.71	36.50	1.82	69.58
CODRNA	0.69	9.07	1.83	41.71
COVTYPE	0.70	8.88	2.00	42.67
ECOLI	0.70	13.15	1.85	44.12
FOURCLASS	0.67	9.04	1.79	42.20
german	0.69	8.94	1.95	42.31
gisette	0.68	8.70	31.72	106.39
GLASS	0.70	4.36	1.73	31.62
ijcnn1	0.67	8.83	1.85	42.13
IMAGESEG	0.78	228.58	2.13	127.59
ionospere	0.67	38.67	2.03	70.18
PIMA	0.76	328.53	2.04	146.07
sonar	0.70	9.03	2.06	40.31

Table 2.3: Time spent by algorithms on several datasets

Dataset	RANDOM	SPCTL2BOOST	BMDR	EXPREPINF
ionospere	0.00076	0.00239	3.76450	1.72880
liver-disorders	0.00062	0.00101	4.09025	1.72851
sonar	0.00068	0.00106	4.29759	1.85681
phishing	0.00063	0.00103	4.66769	1.96604
a1a	0.00062	0.00122	4.72226	2.11856
a2a	0.00063	0.00094	5.16078	2.22566
a3a	0.00067	0.00104	5.19418	2.38561
a4a	0.00079	0.00158	5.55162	2.61149

Table 2.4: Classes of the 20 Newsgroups dataset

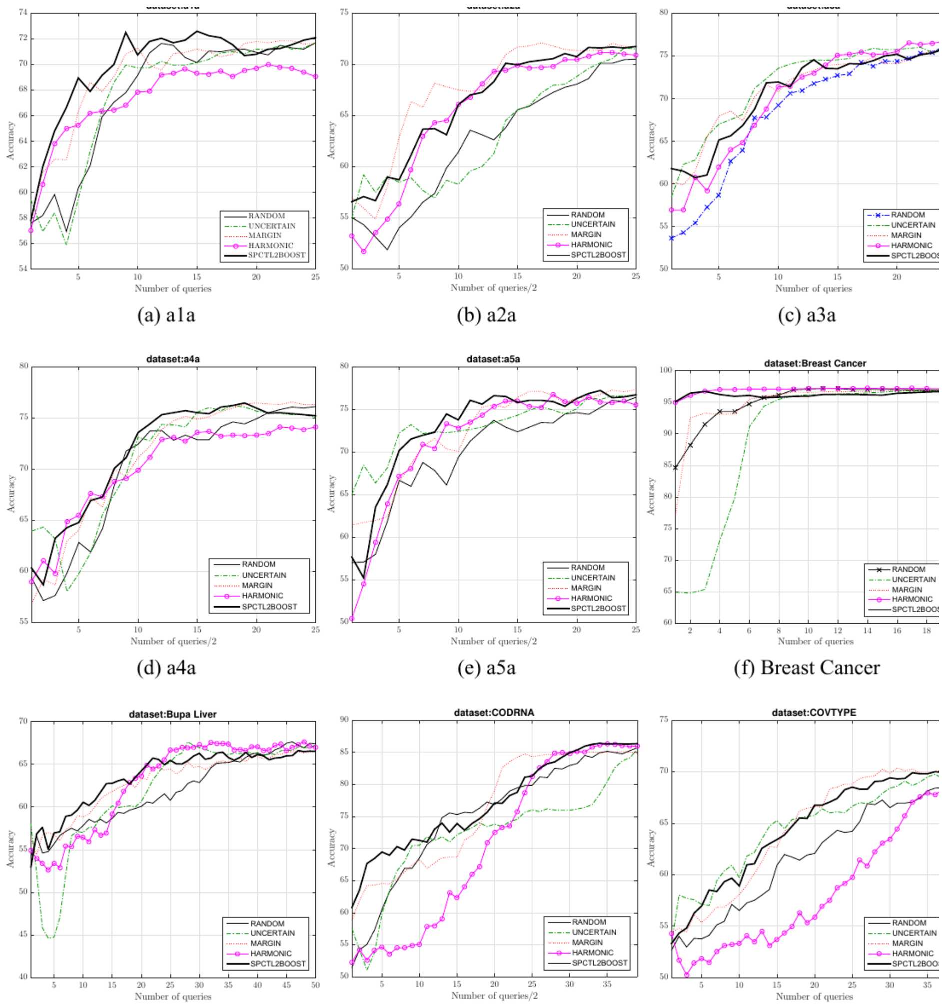
label	class
1	alt.atheism
2	comp.graphics
3	comp.os.ms-windows.misc
4	comp.sys.ibm.pc.hardware
5	comp.sys.mac.hardware
6	comp.windows.x
7	misc.forsale
8	rec.autos
9	rec.motorcycles
10	rec.sport.baseball
11	rec.sport.hockey
12	sci.crypt
13	sci.electronics
14	sci.med
15	sci.space
16	soc.religion.christian
17	talk.politics.guns
18	talk.politics.mideast
19	talk.politics.misc
20	talk.religion.misc



(a) a1a (b) a2a (c) a3a

(d) a4a (e) a5a (f) Breast Cancer

Figure 2.1: Comparison of the performance of the proposed algorithm and other algorithms on various datasets (Part 1)

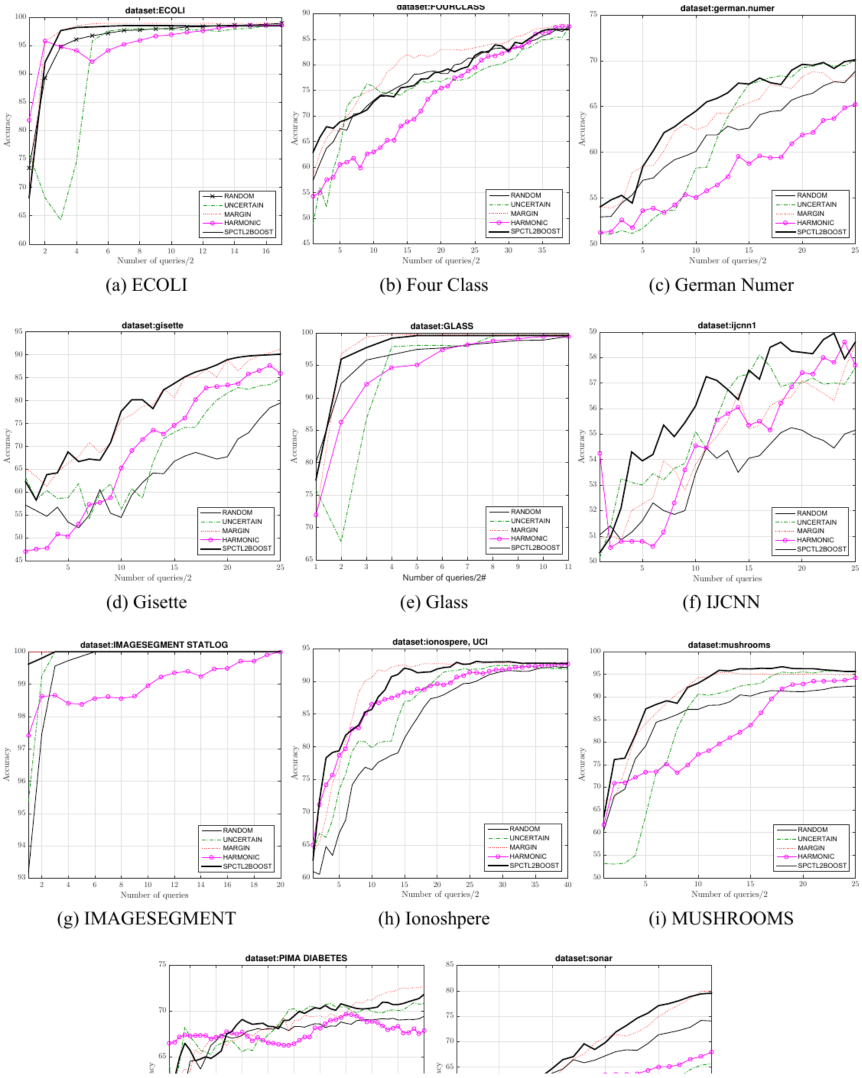


(a) a1a (b) a2a (c) a3a

(d) a4a (e) a5a (f) Breast Cancer

(g) Bupa Liver (h) CODRNA (i) COVTYPE

Figure 2.2: Comparison of the performance of the proposed algorithm and other algorithms on various datasets (Part 2)



(a) ECOLI (b) Four Class (c) German Numer

(d) Gisette (e) Glass (f) IJCNN

(g) IMAGESEGMENT (h) Ionoshpere (i) MUSHROOMS

Figure 2.3: Comparison of the performance of the proposed algorithm and other algorithms on various datasets (Part 3)

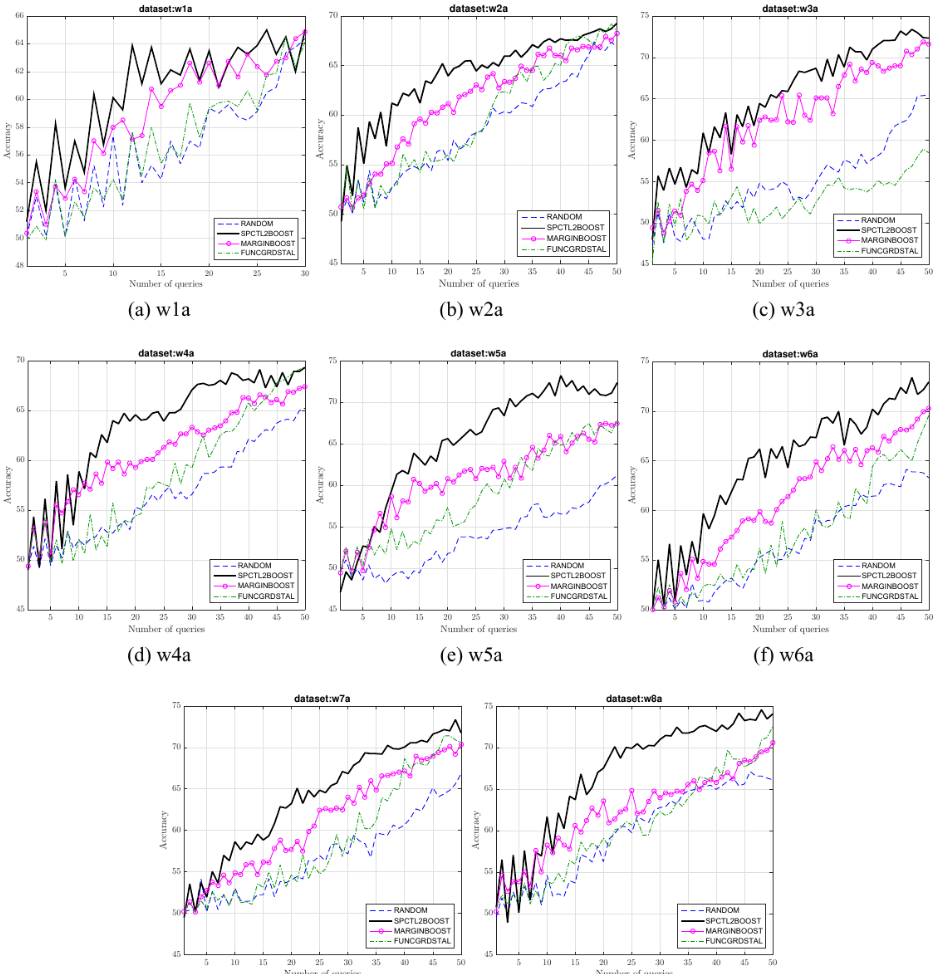
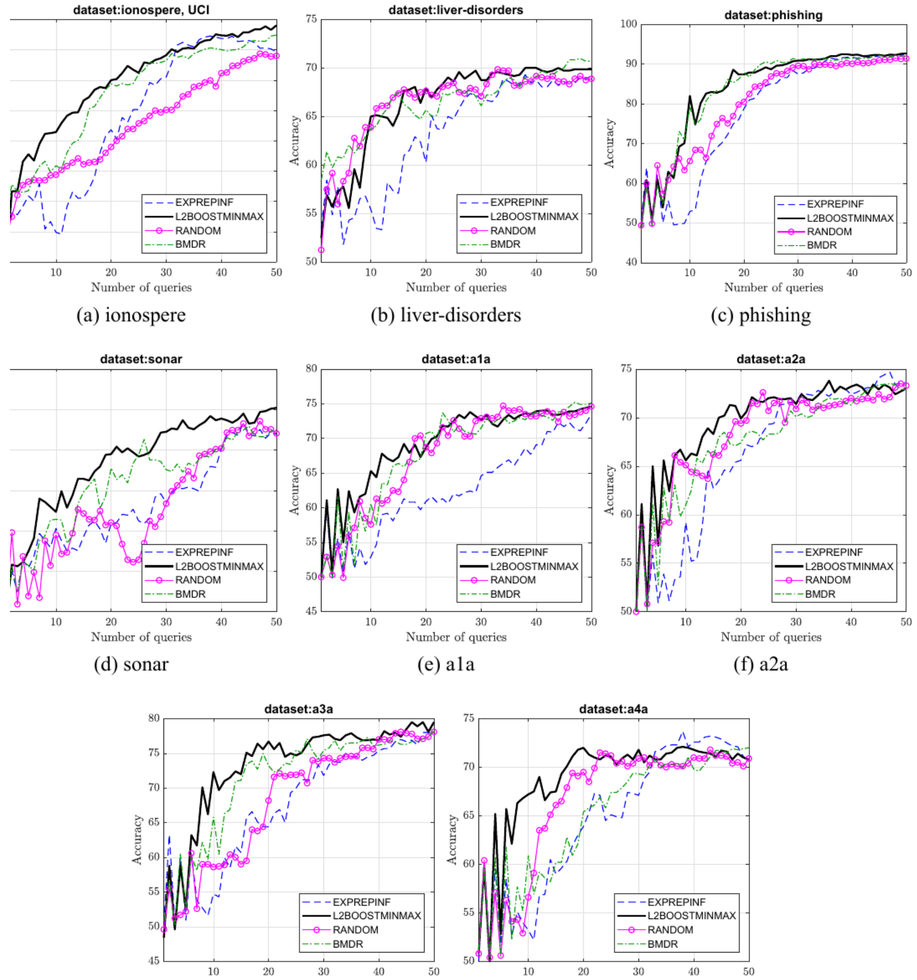


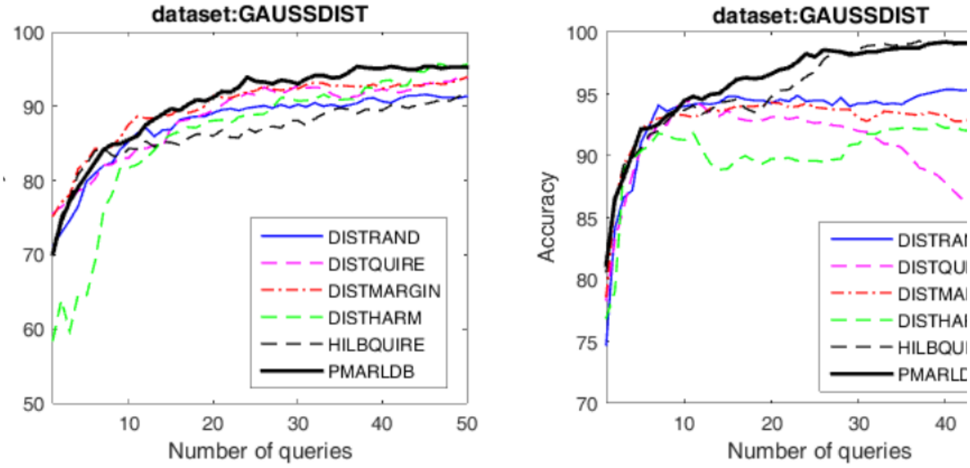
Figure 2.4: Comparison of the proposed method with other boosting methods



(a) ionospere (b) liver-disorders (c) phishing

(d) sonar (e) a1a (f) a2a

Figure 2.5: Comparison of the proposed method with distribution discrepancy methods



ونه با اندازه های $\{5, 100\}$ برای هر توزیع

(ب) نمونه با اندازه 5 برای هرتوزیع

قت طبقه بند در یادگیری فعال روی پایگاه داده GAUSSDIST برای دو حالت با اندازه
توزیع .

Figure 2.6: Comparison of classifier accuracy in active learning on the GAUSSDIST dataset for two cases: different sample sizes and constant sample size for each distribution

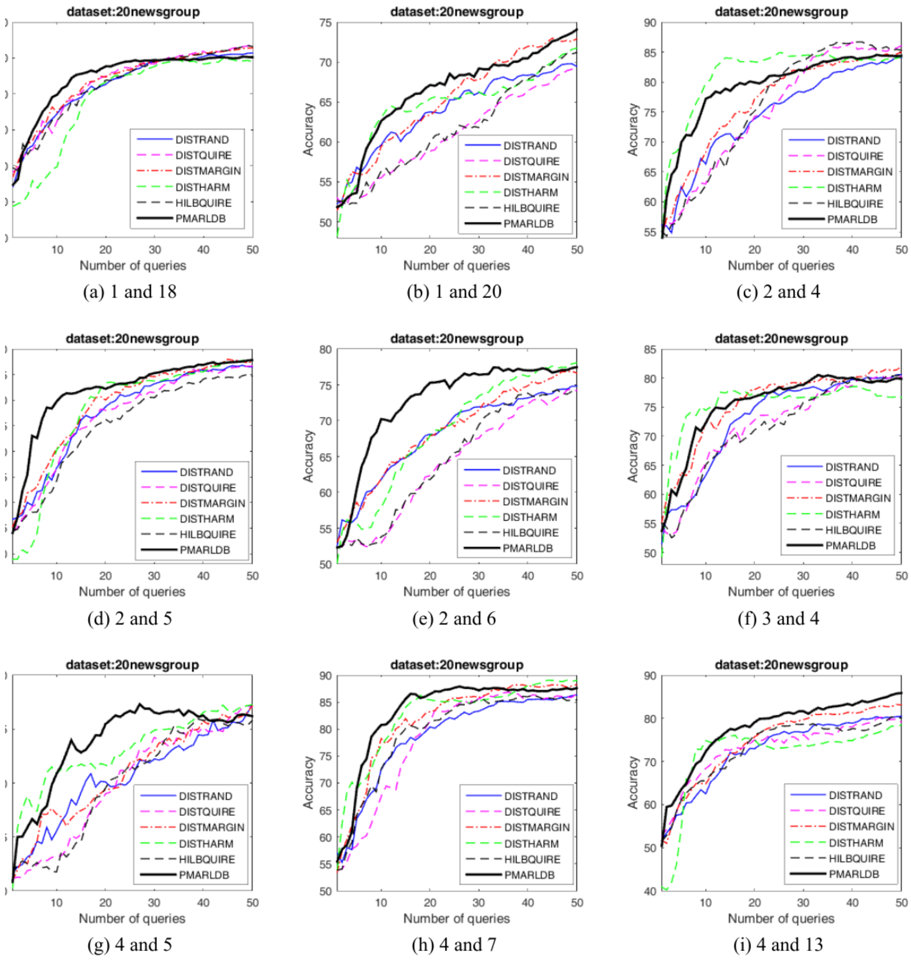


Figure 2.7: Comparison of classification accuracy for a group of 20 Newsgroups datasets (Part 1)

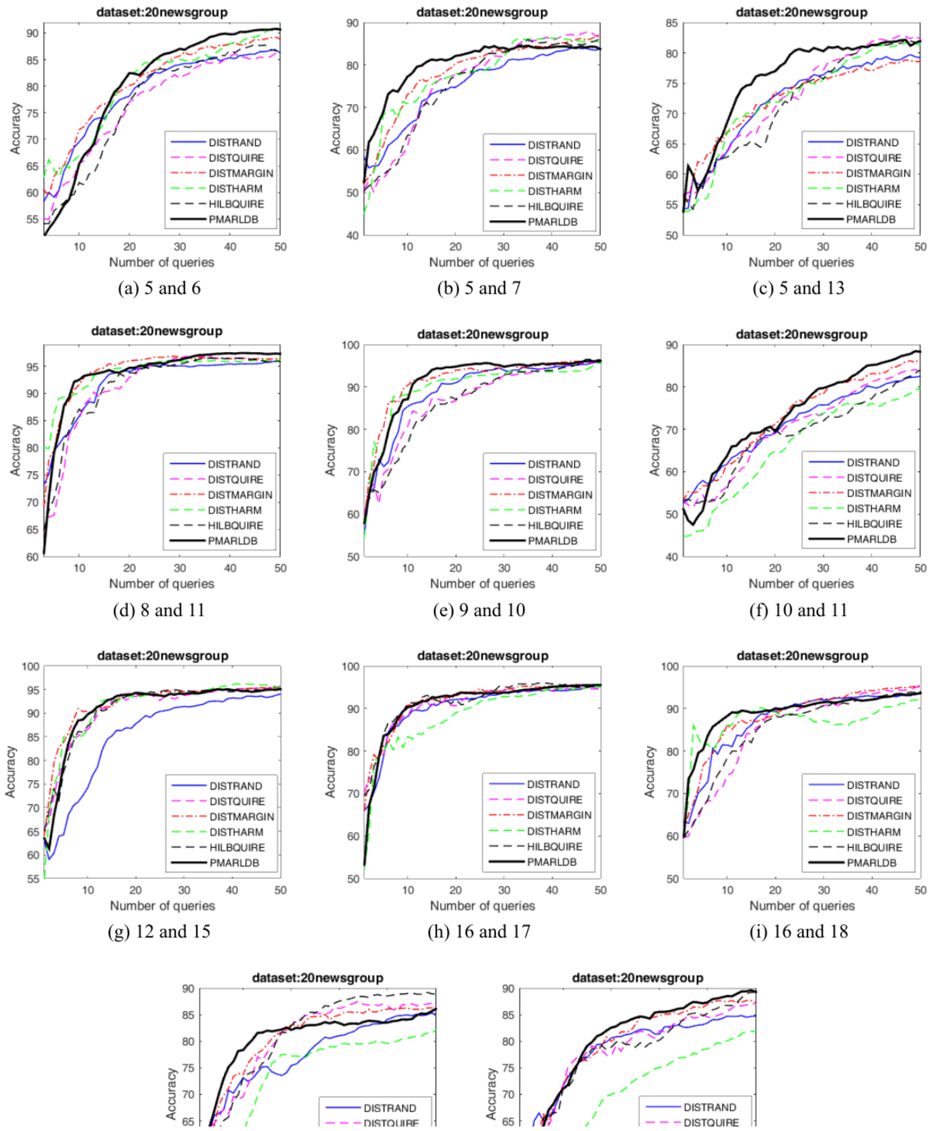
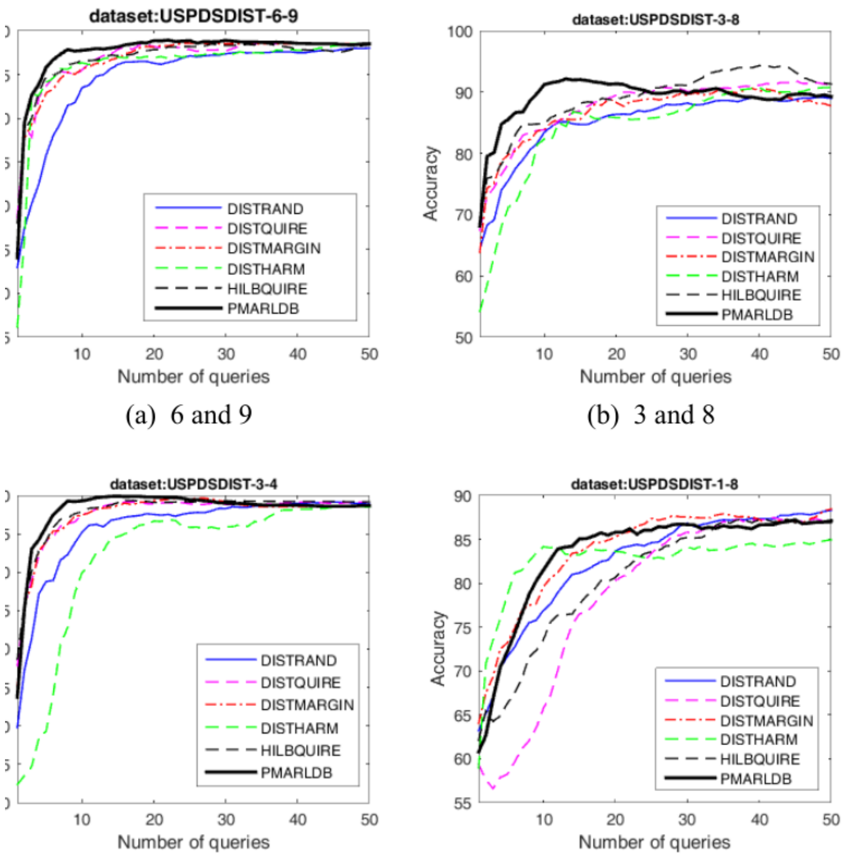


Figure 2.8: Comparison of classification accuracy for another group of 20 Newsgroups datasets (Part 2)



(a) 6 and 9

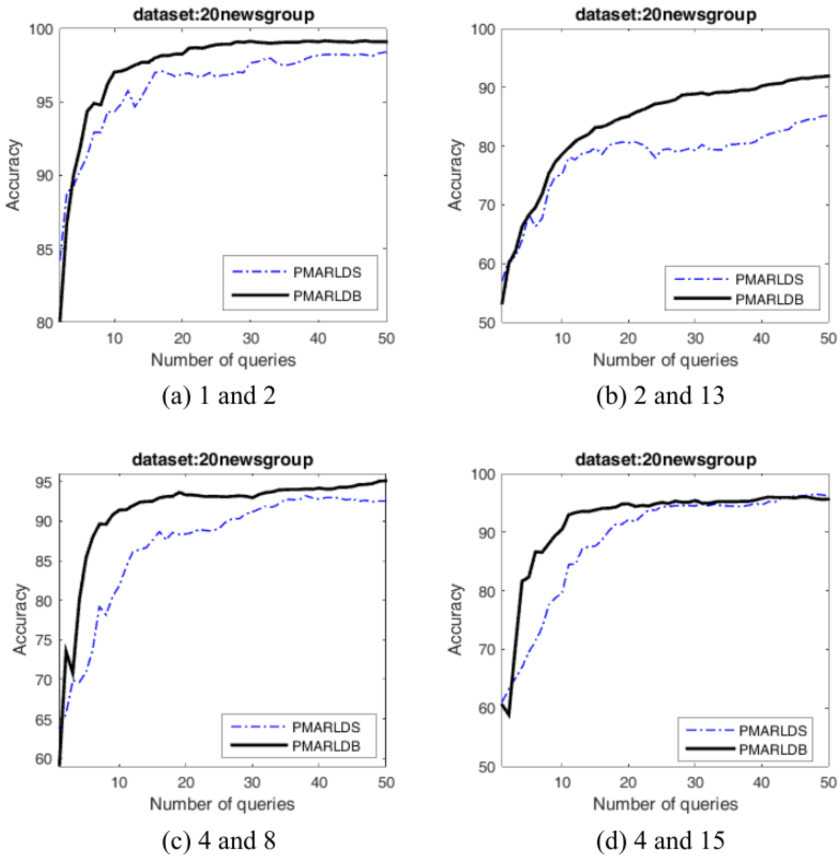
(b) 3 and 8

(c) 3 and 4

(d) 1 and 8

شکل ۹.۴: مقایسه دقت در یادگیری فعال برای پایگاه داده USPDSDIST

Figure 2.9: Comparison of active learning accuracy for the USPSDIST dataset



(a) 1 and 2
(b) 2 and 13
(c) 4 and 8
(d) 4 and 15

Figure 2.10: Comparison of classification accuracy between Bayesian active learning in Hilbert space PMARLDS and PMARLDB on the 20 Newsgroups dataset